

Ops Track

Red Hat K.K.

2025/07/11

The text "Modern Application Development Roadshow" is displayed in a bold, white, sans-serif font, stacked in four lines. It is positioned on the right side of the image, partially overlapping a dark gray diagonal shape that extends from the top right corner towards the bottom left. A small white line-art icon of a cloud with a diamond shape inside is located between the words "Application" and "Development".

**Modern
Application
Development
Roadshow**

本日のスケジュール

長時間となりますが楽しく実施しましょう！

タイムテーブル		開始	終了
1	ごあいさつ (10min)	10:00	10:10
	コンテナ/OpenShift 101 (80 min)	10:10	11:30
	Q&A (10min)	11:30	11:40
	Partner Training Portalのご紹介 (5min)	11:40	11:45
	Lunch休憩 (60～+min)	11:45	13:00
2	ワークショップ環境の払い出し (10min)	13:00	13:10
	OpenShift Ops Track 座学・前半 (50min)	13:10	14:00
	休憩+ハンズオン実施 1~5 (90min)	14:00	15:30
	OpenShift Ops Track 座学・後半 (30min)	15:30	16:00
	ハンズオン実施 5-11(90min) ※前半、後半で分割予定	16:00	17:30
	ラップアップ	17:30	18:00





ワークショップ環境 取得方法

ワークショップ環境取得方法(1)

下記 URL へアクセスしてください

<https://catalog.demo.redhat.com/workshop/vx87kb>

ワークショップ環境取得方法(2)

The screenshot shows the registration page for the 'Modern App Dev Roadshow - Ops Track' workshop on the Red Hat Demo Platform. The page has a dark header with the Red Hat logo and 'Red Hat Demo Platform' text. Below the header is a large banner with the title 'Modern App Dev Roadshow - Ops Track'. Underneath the banner, the text 'Access to Modern App Dev Roadshow - Ops Track' is displayed. The main content area contains a registration form with three fields: 'Email', 'Workshop Password', and a button labeled 'Access this workshop →'. Red boxes highlight each of these three elements, with red arrows pointing from them to Japanese text on the right side of the image.

Red Hat
Demo Platform

Modern App Dev Roadshow - Ops Track

Access to Modern App Dev Roadshow - Ops Track

Email * ⓘ
mkoshimi@redhat.com

Workshop Password * ⓘ
.....

Access this workshop →

メールアドレス

講師からお伝えします

メールアドレスとパスワード入力後
クリック

ワークショップ環境取得方法(3)

 Red Hat
Demo Platform

Instructions for Modern App Dev Roadshow - Ops Track

Messages

OpenShift Console: <https://console-openshift-console.apps.cluster-6sm72.6sm72.sandbox260.opentlc.com>
OpenShift API for command line 'oc' client: <https://api.cluster-6sm72.6sm72.sandbox260.opentlc.com:6443>
Download oc client from <http://mirror.openshift.com/pub/openshift-v4/clients/ocp/stable-4.12/openshift-client-linux.tar.gz>

Access the workshop at <https://dashboard-lab-ocp-cns.apps.cluster-6sm72.6sm72.sandbox260.opentlc.com>
Login with 'kubeadmin' and 'xMKCX-ponge-vrxYT-4CkSp'

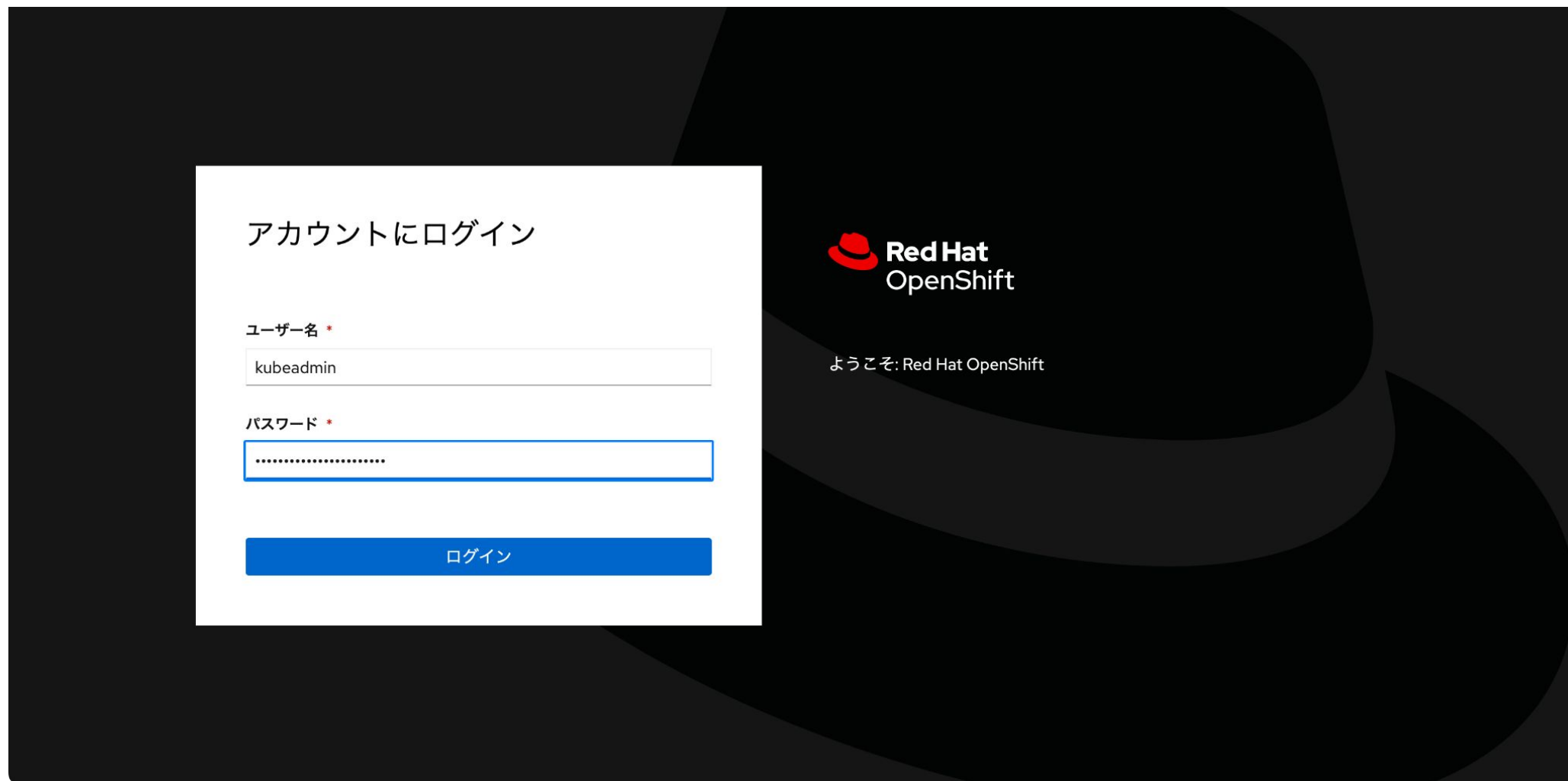
Workshop may not be accessible until rollout finishes shortly.

Your RHACS console is available at:
<https://central-stackrox.apps.cluster-6sm72.6sm72.sandbox260.opentlc.com>
RHACS portal username: admin
RHACS portal password: MTcONjl4

Use the following credentials to deploy in the AWS sandbox account where your Hub is running.

ワークショップ環境の URL
ユーザ名とパスワード

ワークショップ環境取得方法(4)



The image shows a Red Hat OpenShift login interface. On the left, a white modal box titled 'アカウントにログイン' (Login to account) contains a login form. The form has two input fields: 'ユーザー名 *' (Username *) with the value 'kubeadmin' and 'パスワード *' (Password *) with masked characters. Below the fields is a blue button labeled 'ログイン' (Login). On the right, the background is dark with the 'Red Hat OpenShift' logo and the text 'ようこそ: Red Hat OpenShift' (Welcome: Red Hat OpenShift).

アカウントにログイン

ユーザー名 *

kubeadmin

パスワード *

ログイン

Red Hat OpenShift

ようこそ: Red Hat OpenShift

ワークショップ環境取得方法(5)

Authorize Access

Service account dashboard-user in project lab-ocp-cns is requesting permission to access your account (kube:admin)

Requested permissions

☒ **user:info**

Read-only access to your user information (including username, identities, and group membership)

You will be redirected to https://dashboard-lab-ocp-cns.apps.cluster-6sm72.6sm72.sandbox260.opentlc.com/oauth_callback

Allow selected permissions

Deny

選択

ワークショップ環境取得方法(6)

Modern App Dev Roadshow -
Ops Track

WORKSHOP
MODULES

- 演習環境の概要
 - OpenShift クラスターのインストールの検証
 - アプリケーション管理の基礎
 - アプリケーションストレージの基礎
 - MachineSets, Machines, and Nodes
 - インフラストラクチャノードとOperator
 - OpenShift ログ集約
 - 外部認証プロバイダ(LDAP)の設定
 - OpenShift Monitoring
 - プロジェクト・リクエスト・テンプレートとクォータ制限

■ 演習環境の概要

OpenShift クラスタ構成

このワークショップは、Amazon Web Servicesで稼働するOpenShift4クラスタを使って操作します。

OpenShift 4のインストールは事前に完了しています。OpenShift4クラスタは基本的にすべてデフォルトに設定されており、以下のようなクラスタ構成になっています。

- 3 Master ノード
- 2 または 3 Worker ノード
- 1 Bastion Host (踏み台ホスト)

ターミナルとWebコンソールの使い方

このワークショップは、[Homeroom](#) というツールを利用しています。Homeroomでは、画面左側に(まさに今読んでいる)演習ガイドが表示され、画面右側にはターミナルが表示

Terminal Console

[~] \$

Modern Application Development Roadshow Ops Track

- Module 1
 - OpenShift管理者の基礎知識
- Module 2
 - Red Hat Advanced Cluster Security for Kubernetesを用いたクラウド環境のセキュリティ強化
- Module 3
 - Red Hat Advanced Cluster Management for Kubernetesを用いたマルチクラスター管理



Module 1

OpenShift管理者の基礎知識

OpenShift管理者の基礎知識

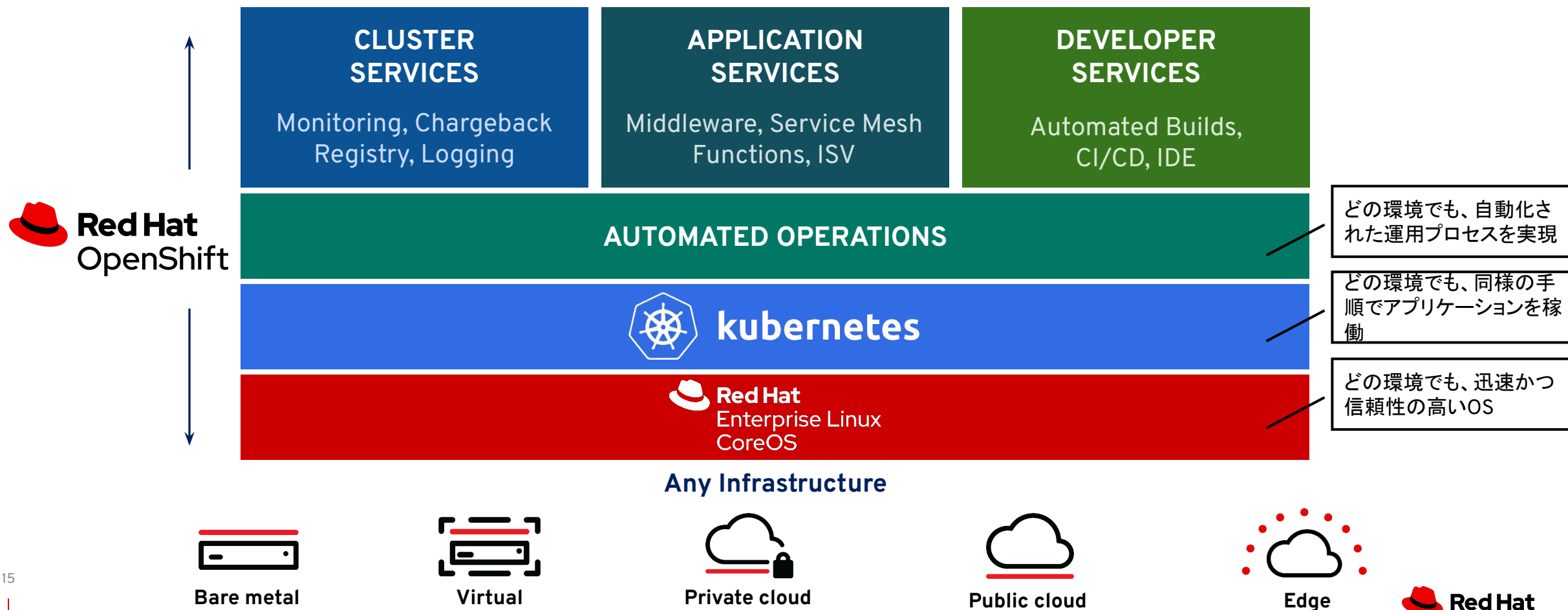
0. 演習環境の概要

1. OpenShiftクラスタのインストールの検証
2. アプリケーション管理の基礎
3. アプリケーションストレージの基礎
4. MachineSets, Machines, and Nodes
5. インフラストラクチャノードとOperator
6. 外部認証プロバイダ(LDAP)の設定
7. OpenShift Monitoring
8. プロジェクト・リクエスト・テンプレートとクォータ/制限
9. OpenShift の Network Policy ベースの SDN
10. Projectのセルフプロビジョニングの無効化
11. クラスタリソースのクォータ
12. TaintとToleration

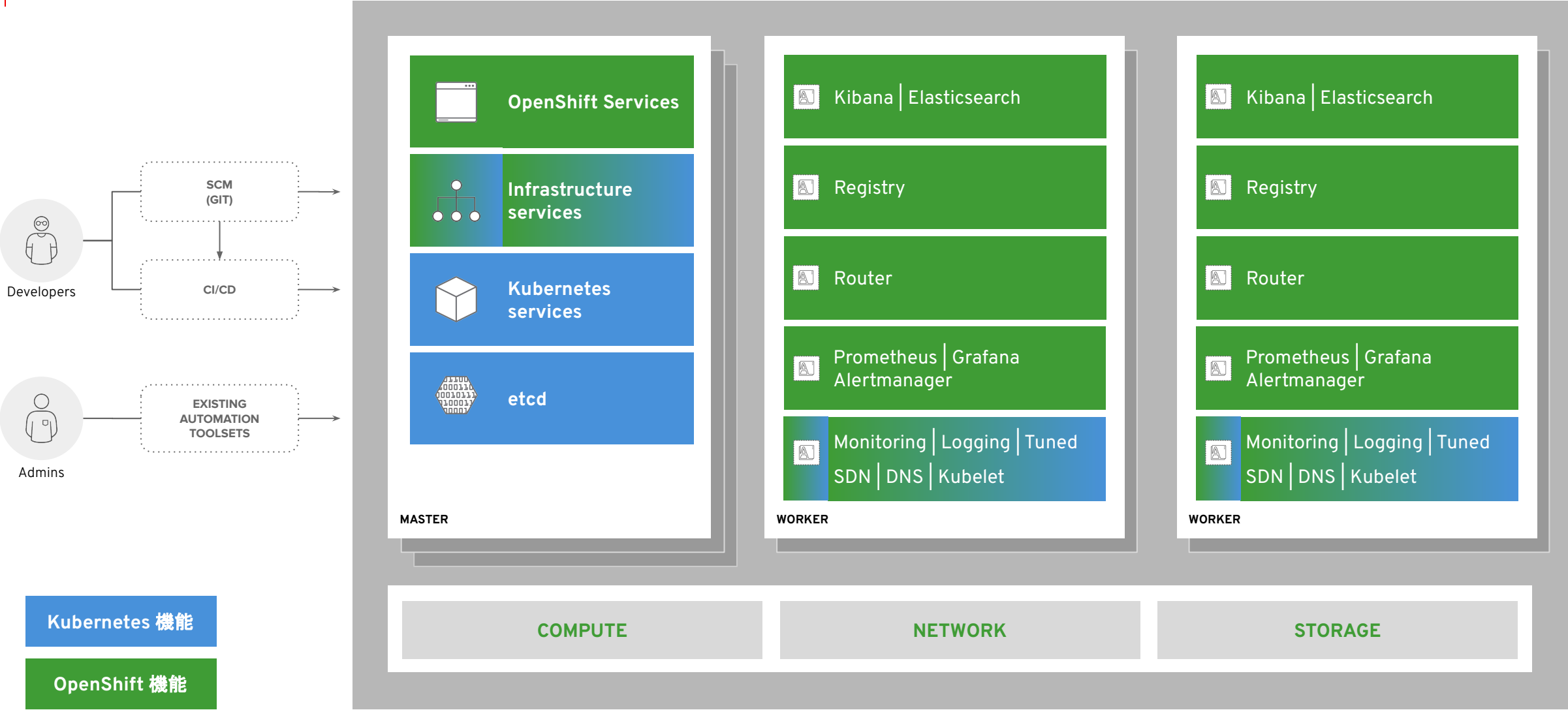
OpenShift アーキテクチャ

OpenShiftはKubernetesを補完する付加価値機能を提供

運用プロセスを含めたポータビリティの実現



OpenShift のアーキテクチャ



Note:

コンテナ

コンテナは最小のコンピュータユニットです。
コンテナはコンテナイメージによって作成されます

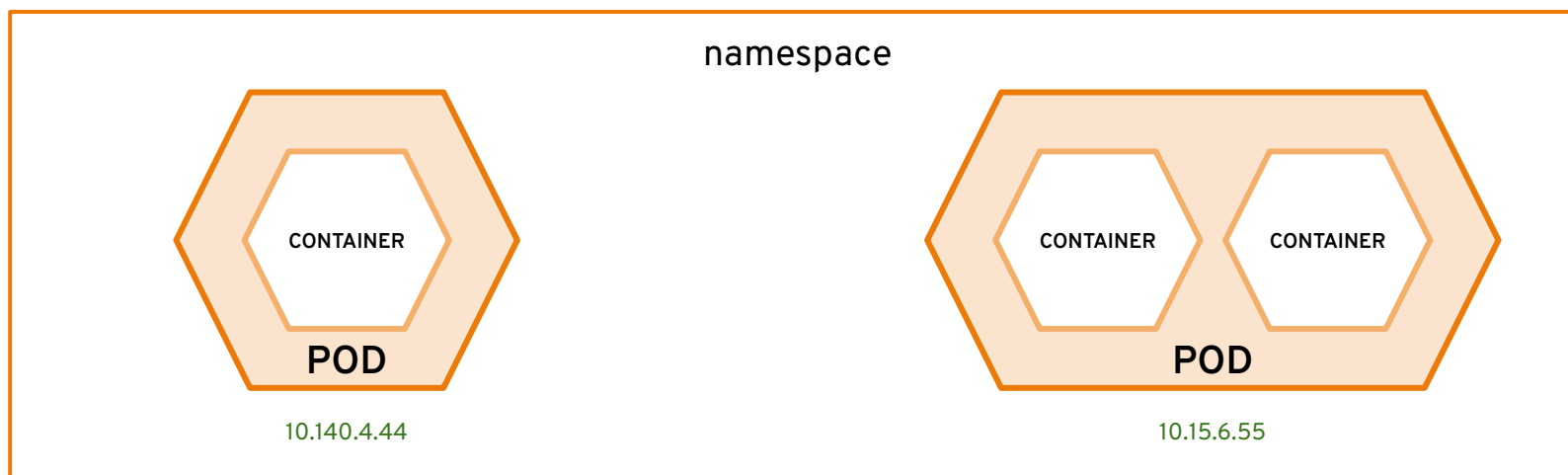


Note:

Pod

コンテナは、デプロイと管理の単位であるPodsに包含されています。

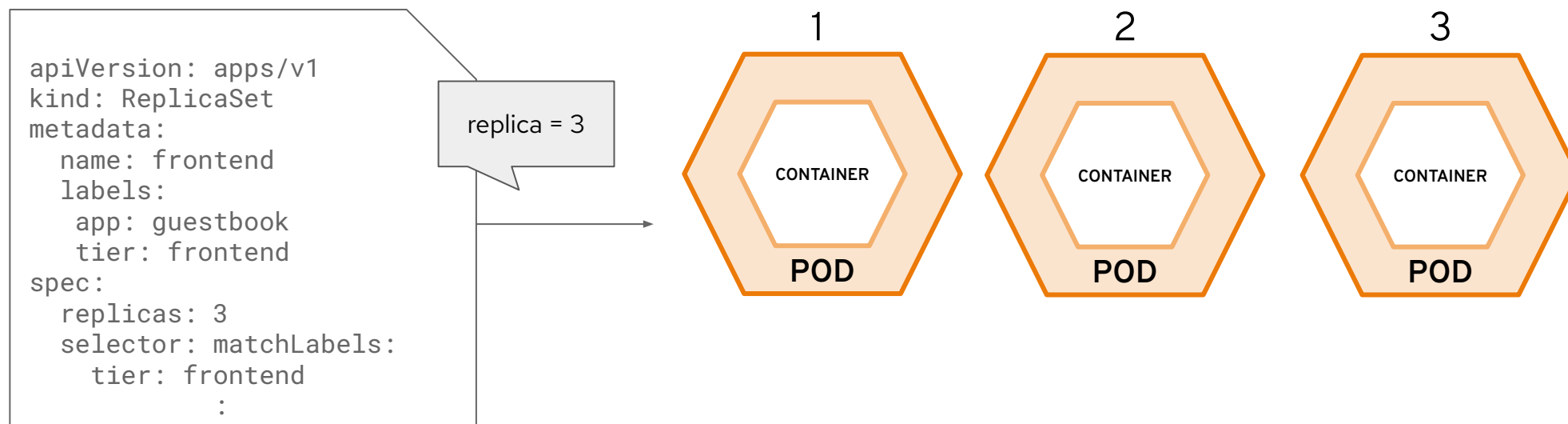
- ・コンテナ内で複数プロセスを実行するのは推奨されない。
- ・Pod内のコンテナは名前スペースを共有 (プロセス間はlocalhostで通信可)



Note:

ReplicaSet

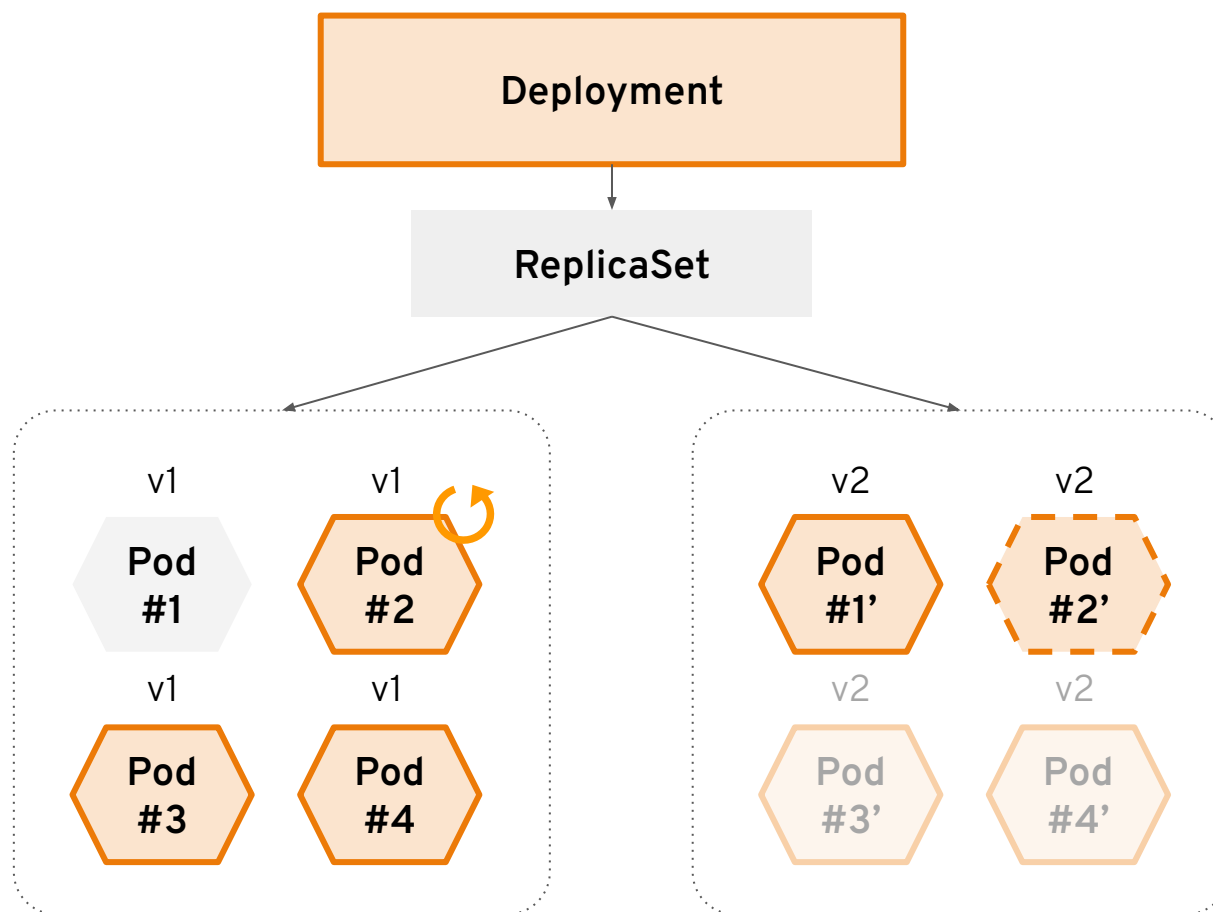
ReplicaSets指定された数のPodが常に稼働していることを保証します。



Note:

Deployment

Podの新バージョンをロールアウトする方法を定義します。

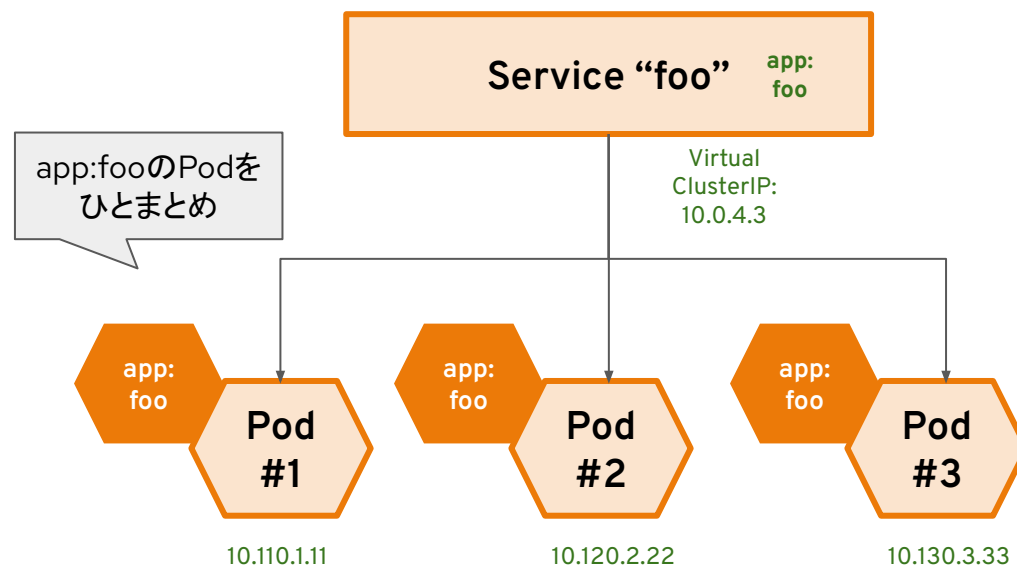


Note:

Services

servicesは、Pod間の以下の機能を提供します。

- ・サービスディスカバリー: サービス名で対象のPodへの通信が可能(無いとIPアドレスのみ)
- ・内部ロードバランシング: サービス名で対象の全てのPodにロードバランシング可能



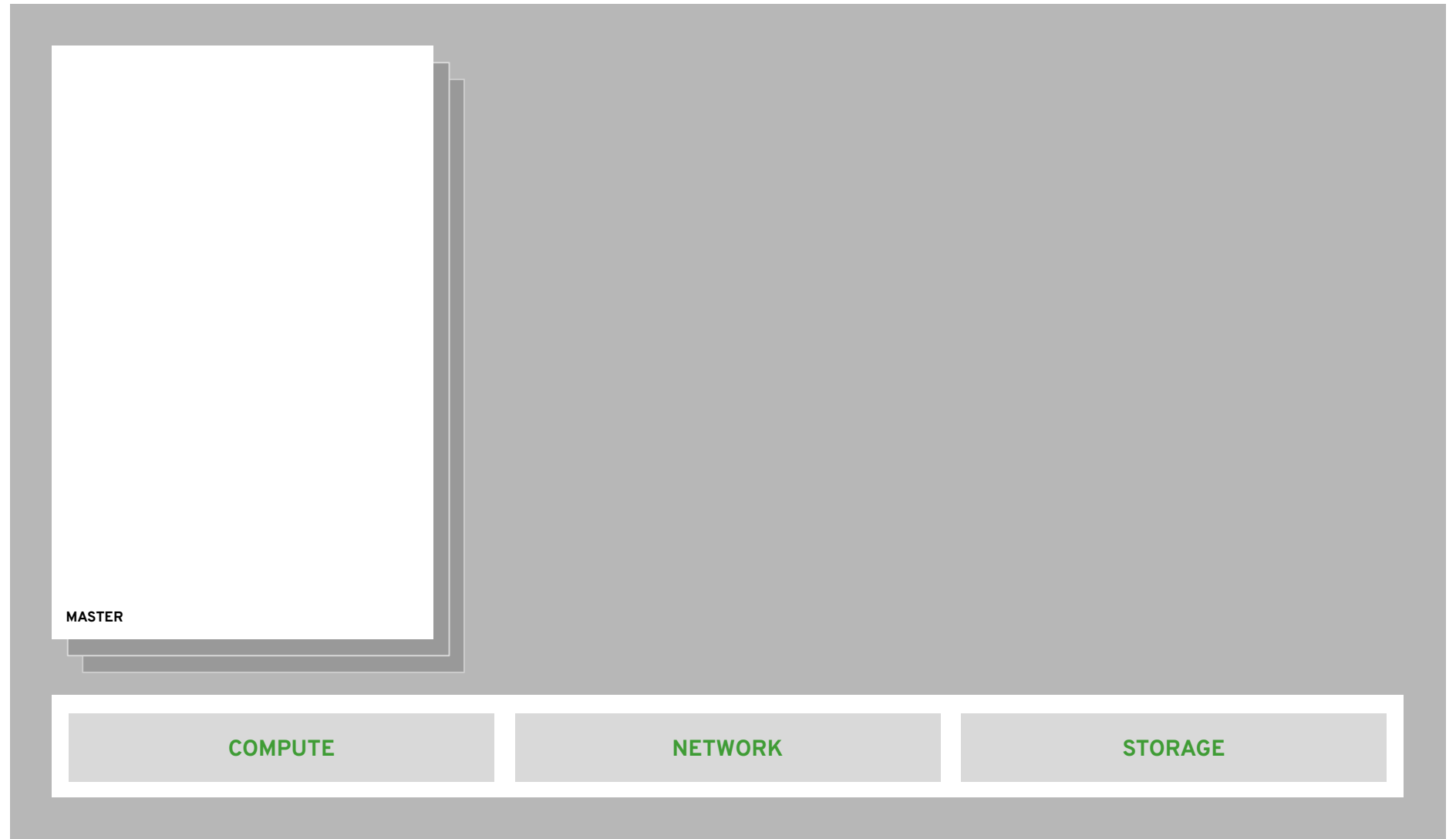
OpenShift のアーキテクチャ - クラスタ構成

COMPUTE

NETWORK

STORAGE

OpenShift のアーキテクチャ - クラスタ構成



OpenShift のアーキテクチャ - クラスタ構成



Note:

Kubernetesの基本コンポーネント

Kubernetesクラスタには2つのコンポーネントがあります。

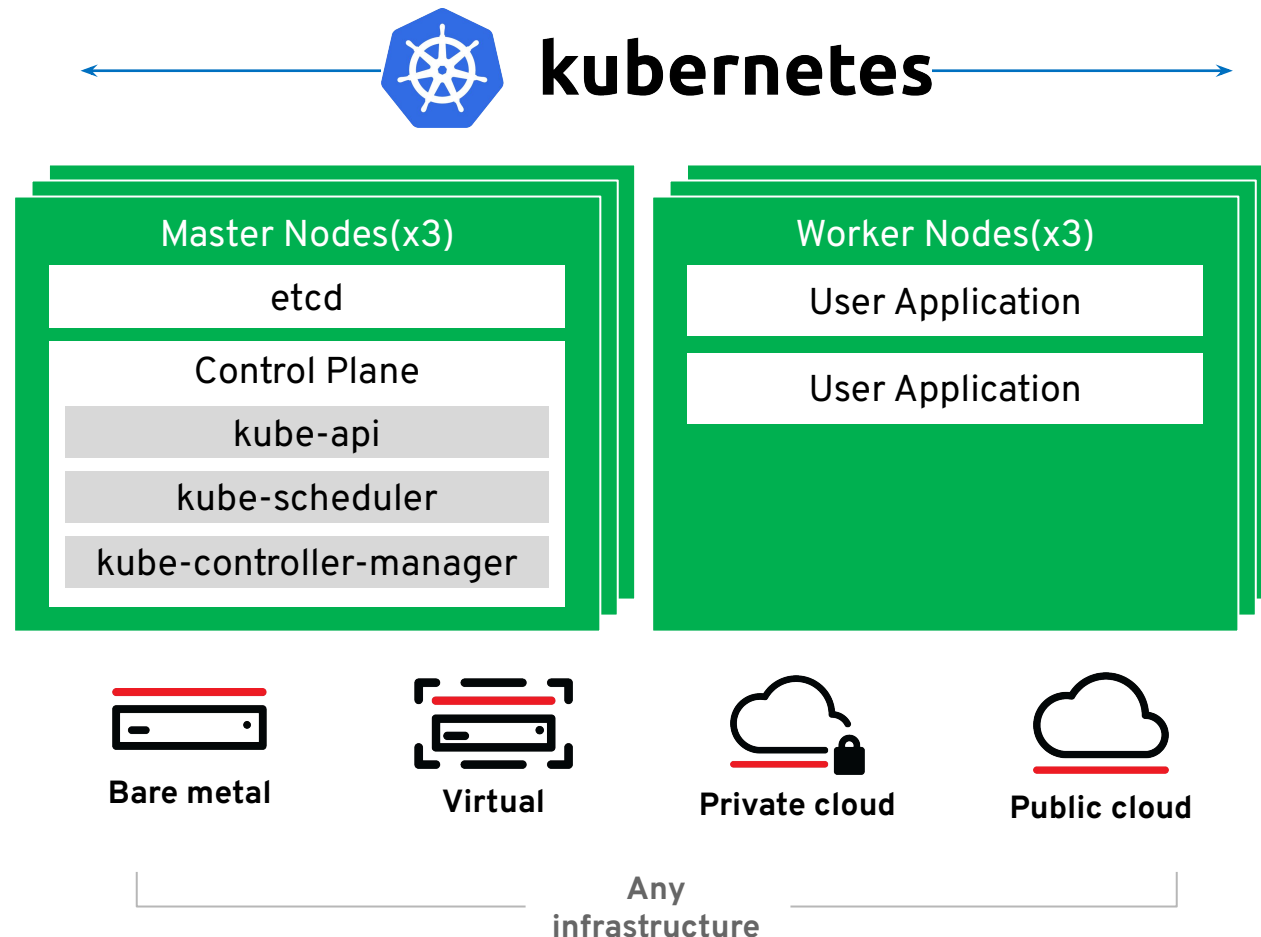
- Master Node (コントロールプレーン)
- Worker Node (コンピュータマシン)

各ノードは独立したLinux環境であり、物理マシンでも仮想マシンでもかまいません。

OpenShiftは上記2つに加えて、さらに

- Infra Node (コンピュータマシン)

を任意で加えることができます



OpenShiftのオペレーティングシステム

コントロールプレーンのOS

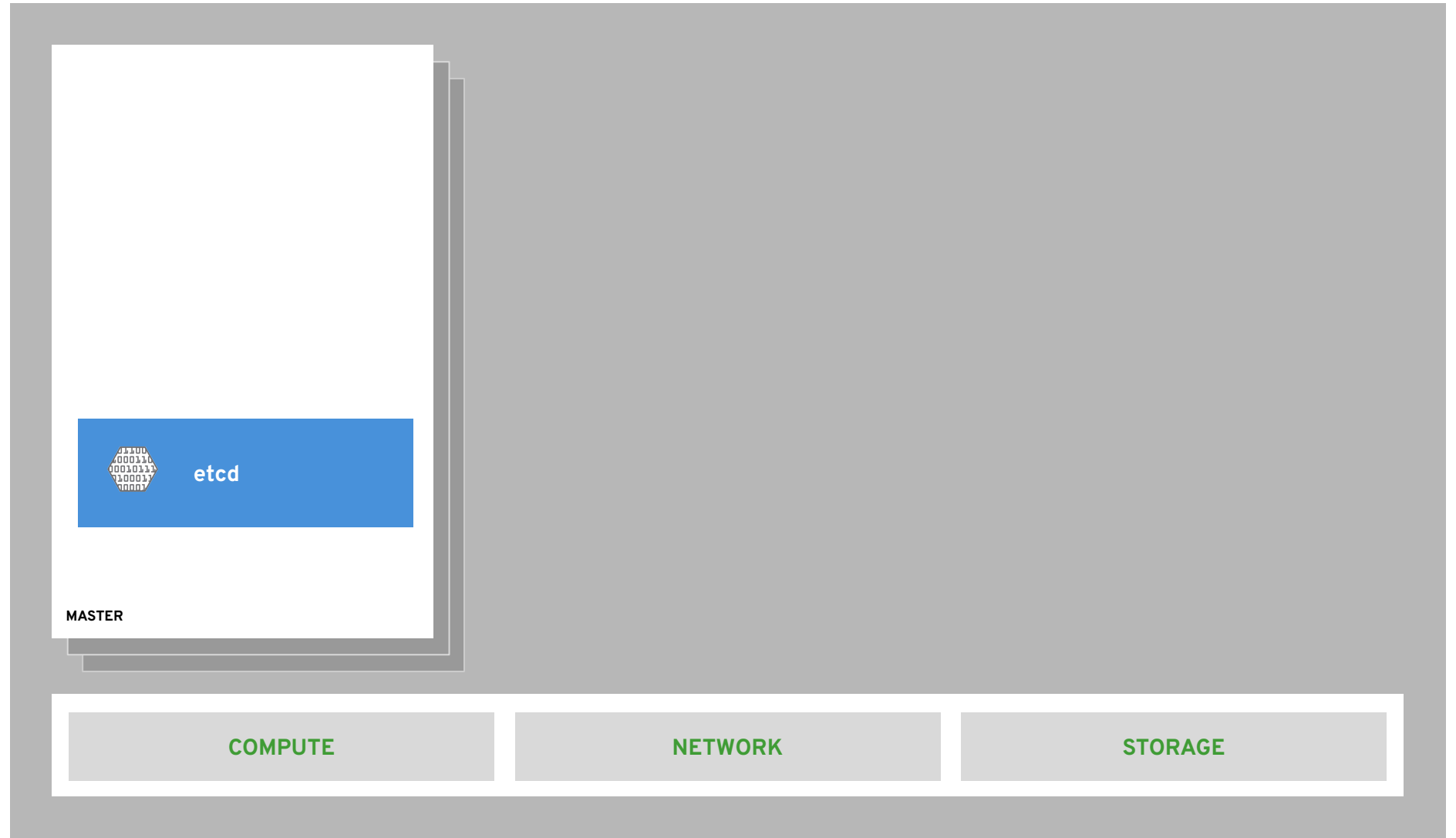
コンピュータのOS

コントロールプレーンのOS

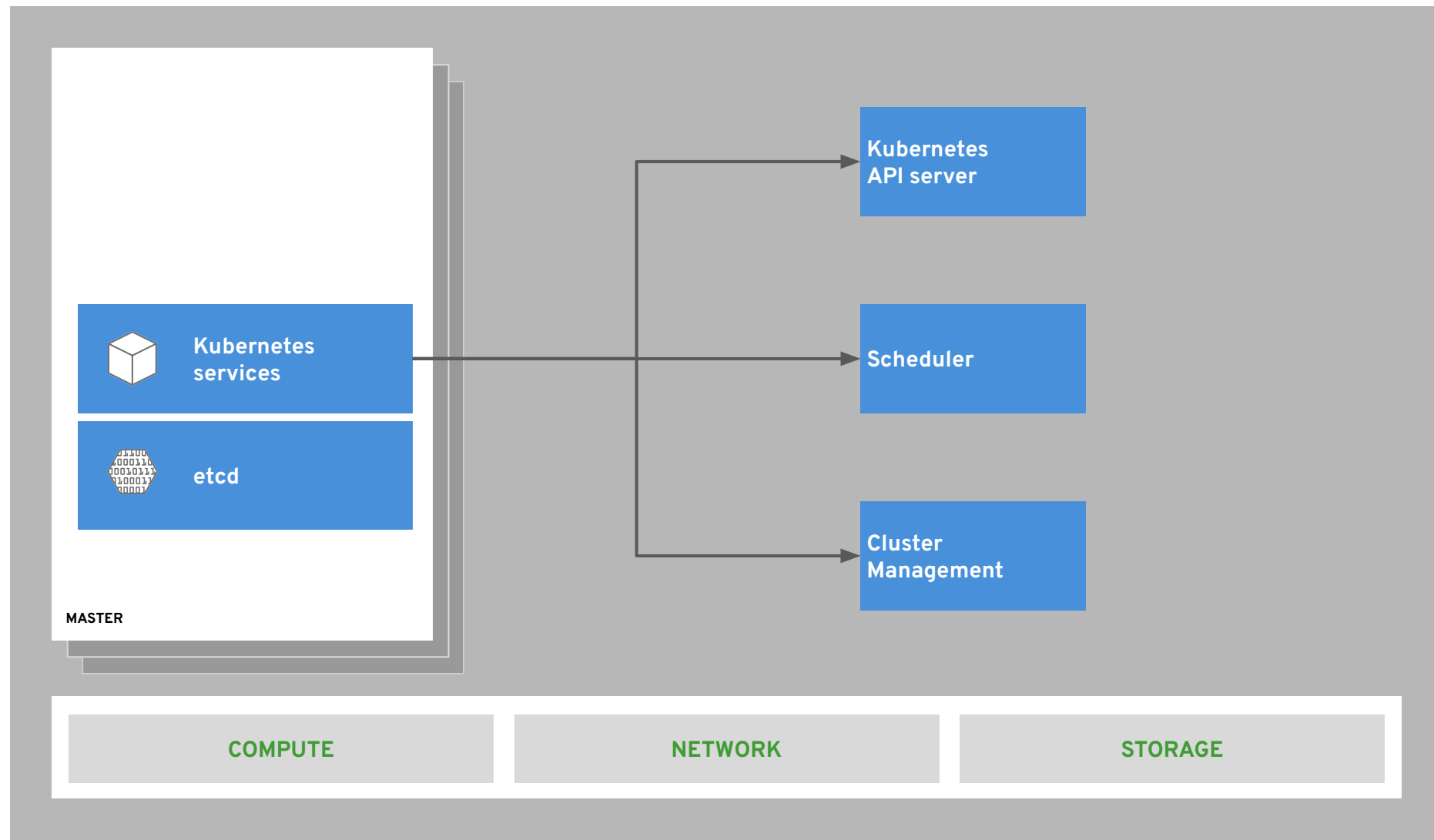
コンピュータのOS

V00000000

OpenShift のアーキテクチャ - etcd によるステート管理



OpenShift のアーキテクチャ - Kubernetes のコアコンポーネント

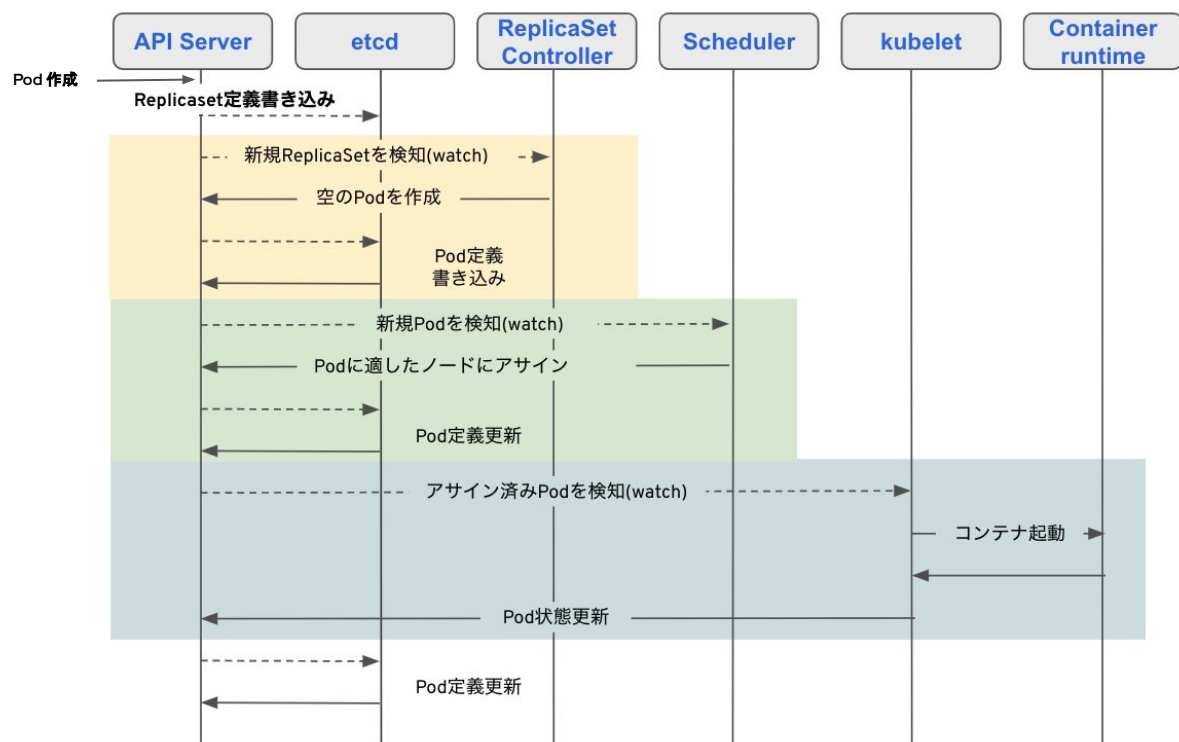


Note:

Kubernetesの基本動作

etcd: Kubernetesのリソース情報を確保する分散キーバリューストア

etcdに書かれたステートを正とし、その状態をコンテナ基盤上で維持し続ける



API Server:

KubernetesへのAPIリクエストをREST経由で受付するコンポーネント

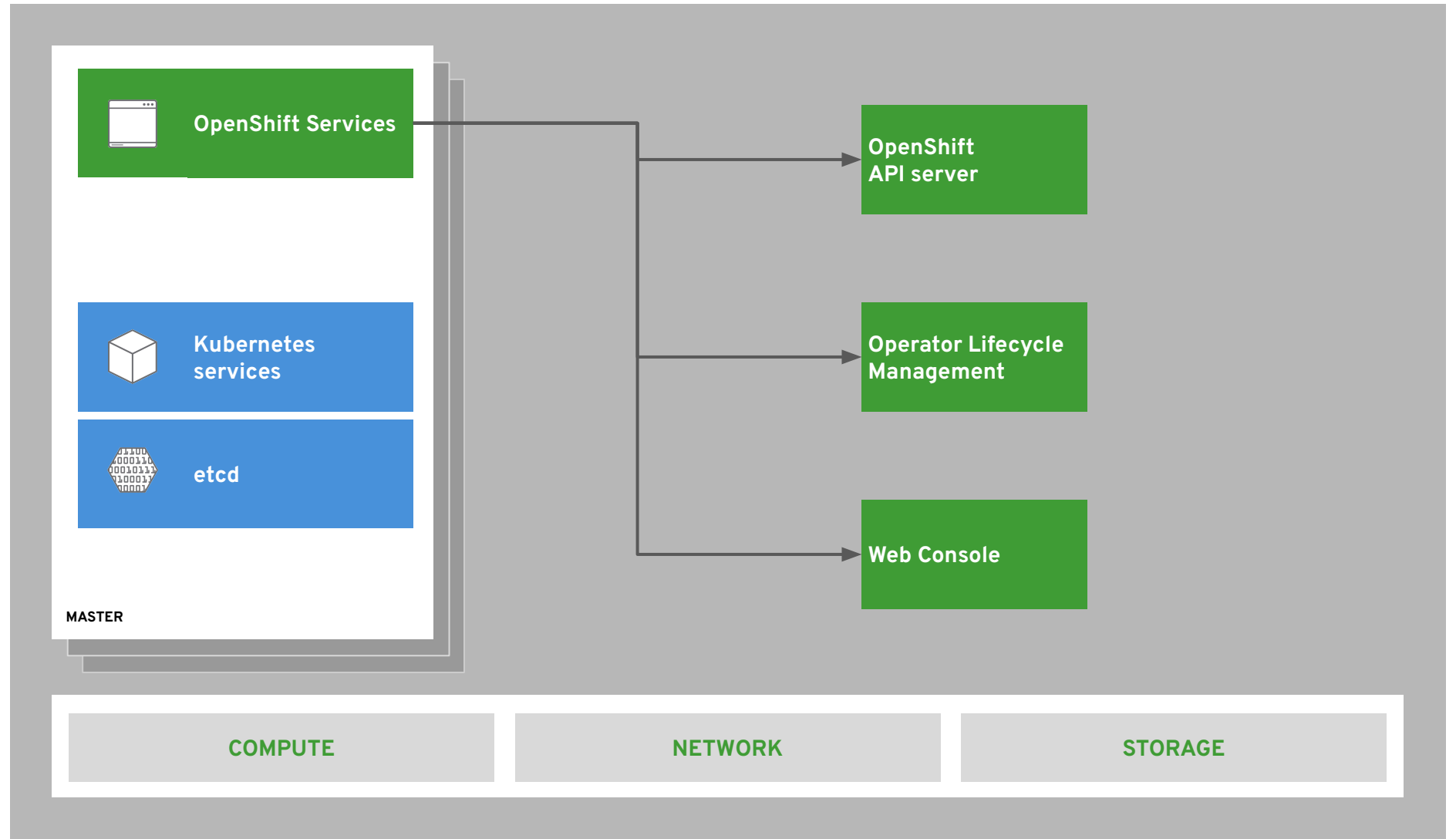
Scheduler:

Podが新規に作成された時やノード障害でPodが再作成されたときに適切なコンピュータードに割り当てるコンポーネント

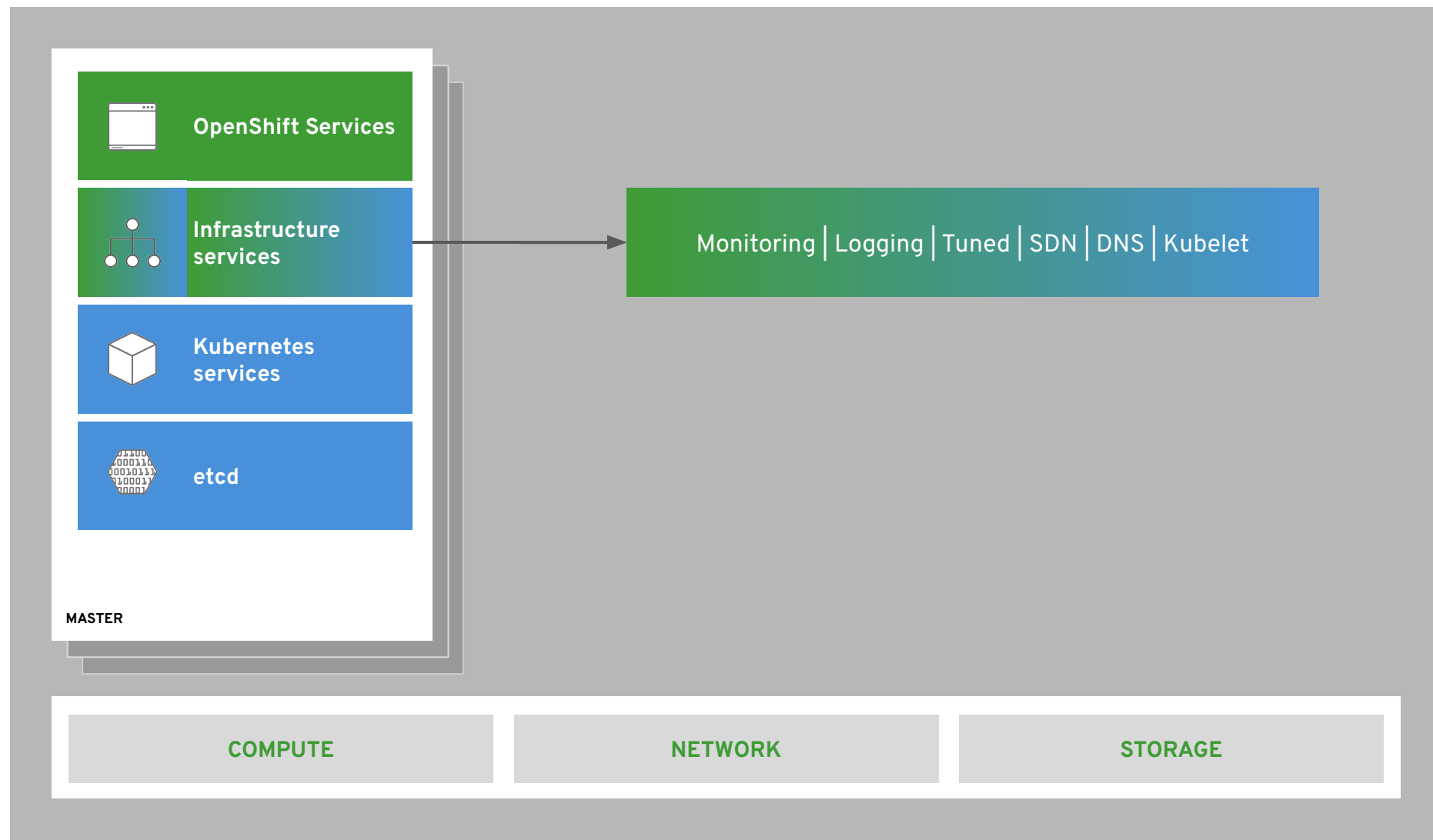
Kubelet:

etcdに保存されている構成から、担当ノードで実行されるべきコンテナを見つけ、Podの起動を行うコンポーネント。Podが望ましい状態で稼働していることも確認します。

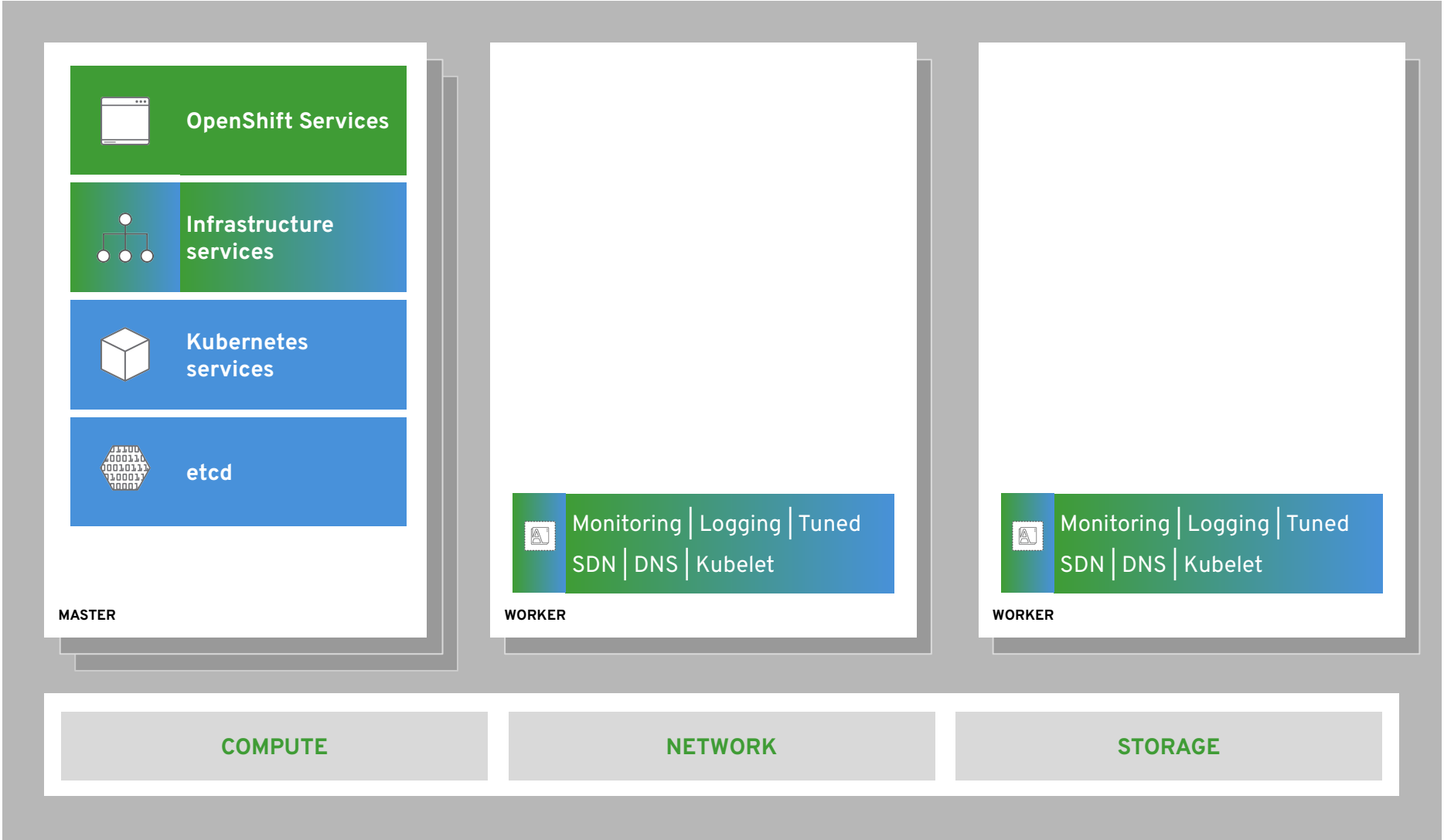
OpenShift のアーキテクチャ - OpenShift のコアコンポーネント



OpenShift のアーキテクチャ - インフラを支えるコンポーネント



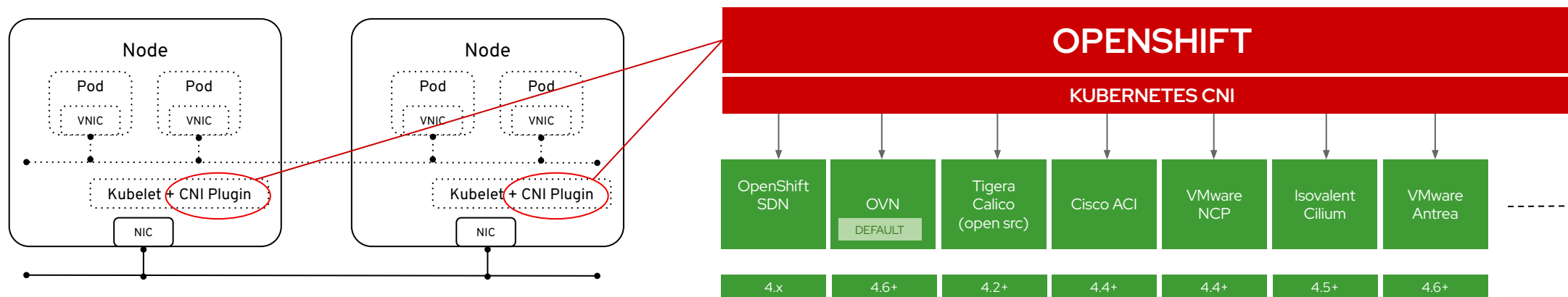
OpenShift のアーキテクチャ - インフラを支えるコンポーネント



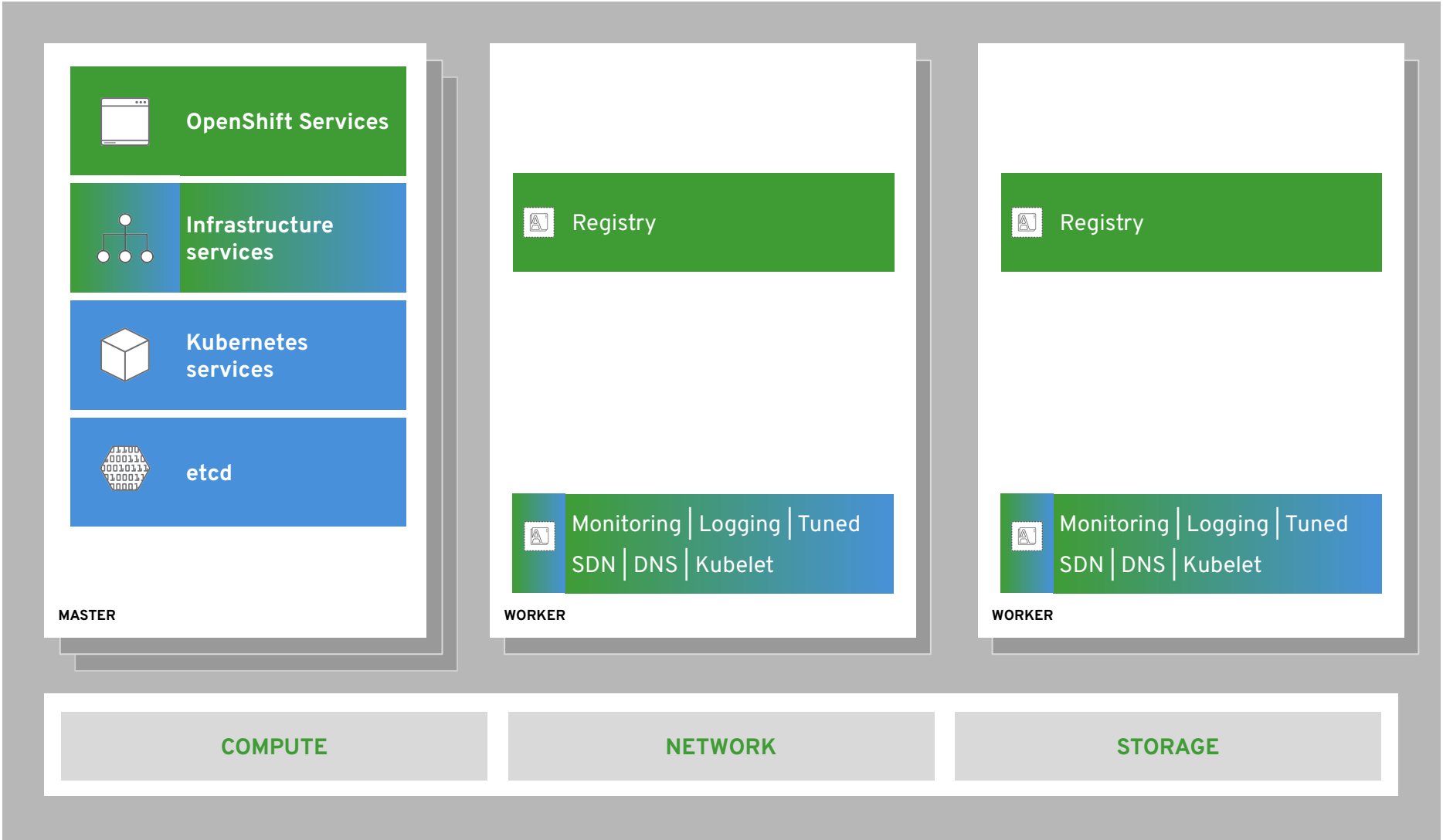
Note:

OpenShift Networking Plug-ins

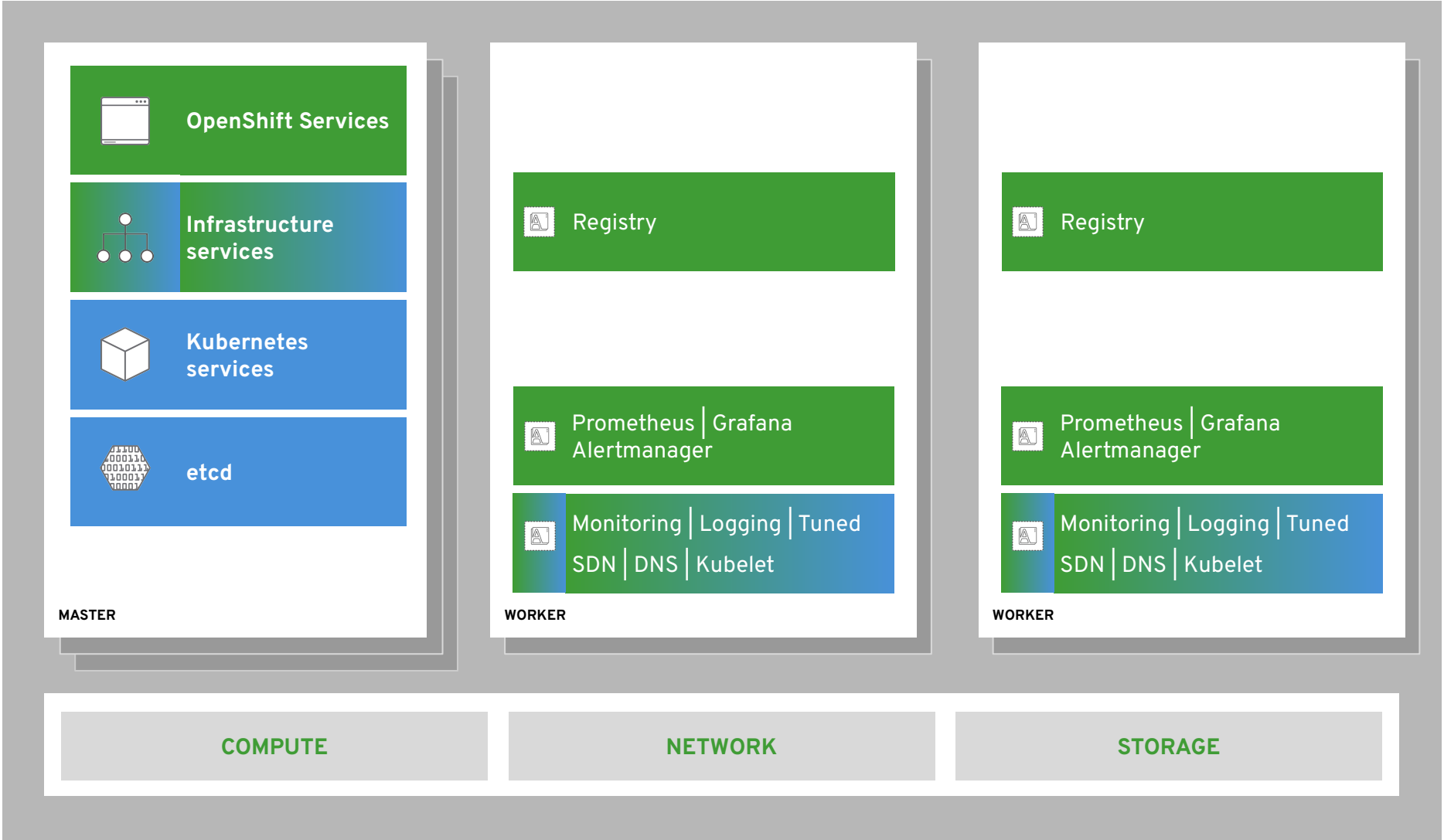
- CNI (Container Network Interface) というインタフェースを定義し、そこに3rd Partyが作成した仮想ネットワーク用のプラグインを使用する。OpenShift では、「OpenShift SDN」と「OVN」と呼ばれるプラグインを提供
- 4.10 からデフォルトはOVN



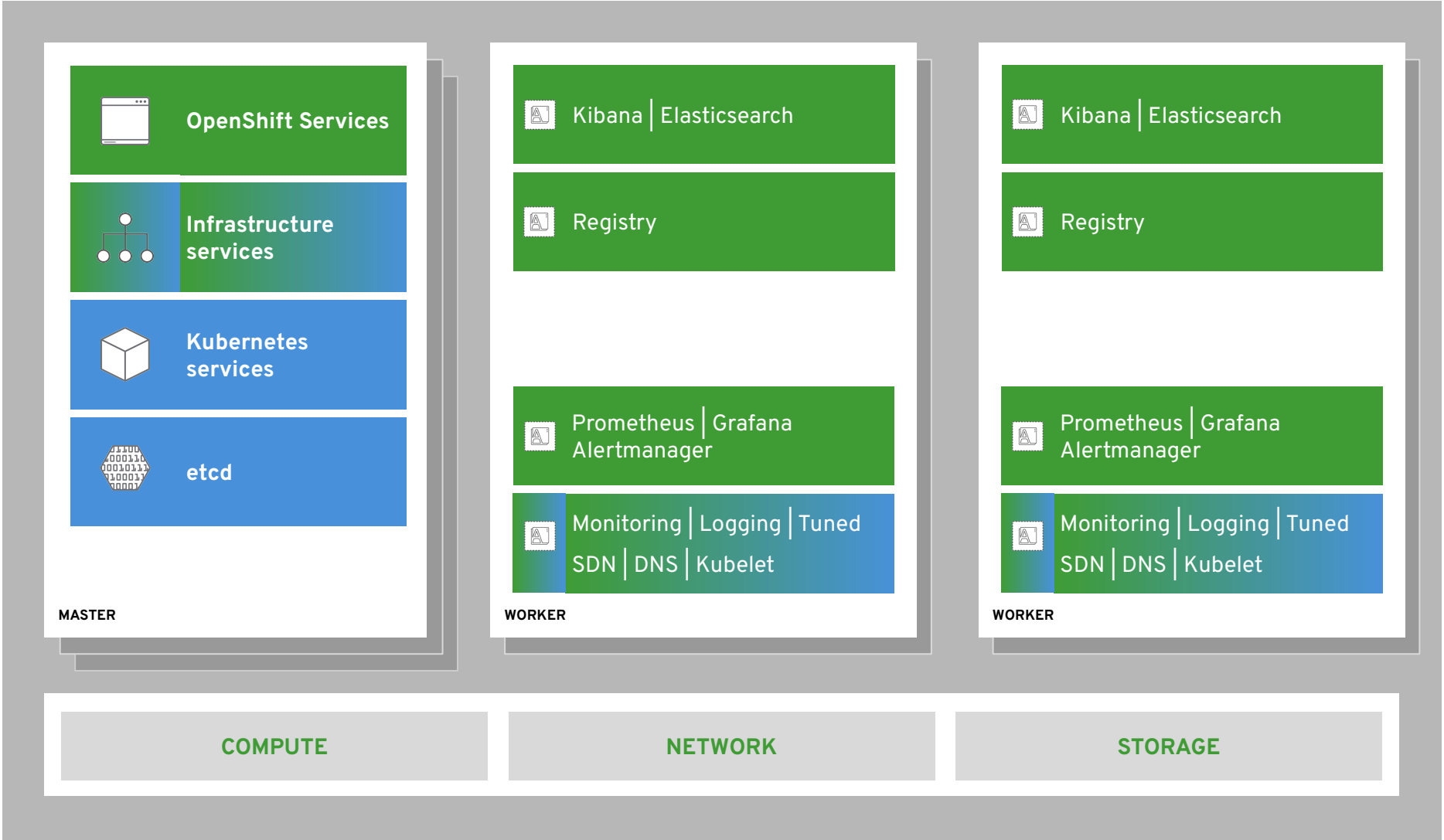
OpenShift のアーキテクチャ - OpenShift 内部のコンテナイメージレジストリ



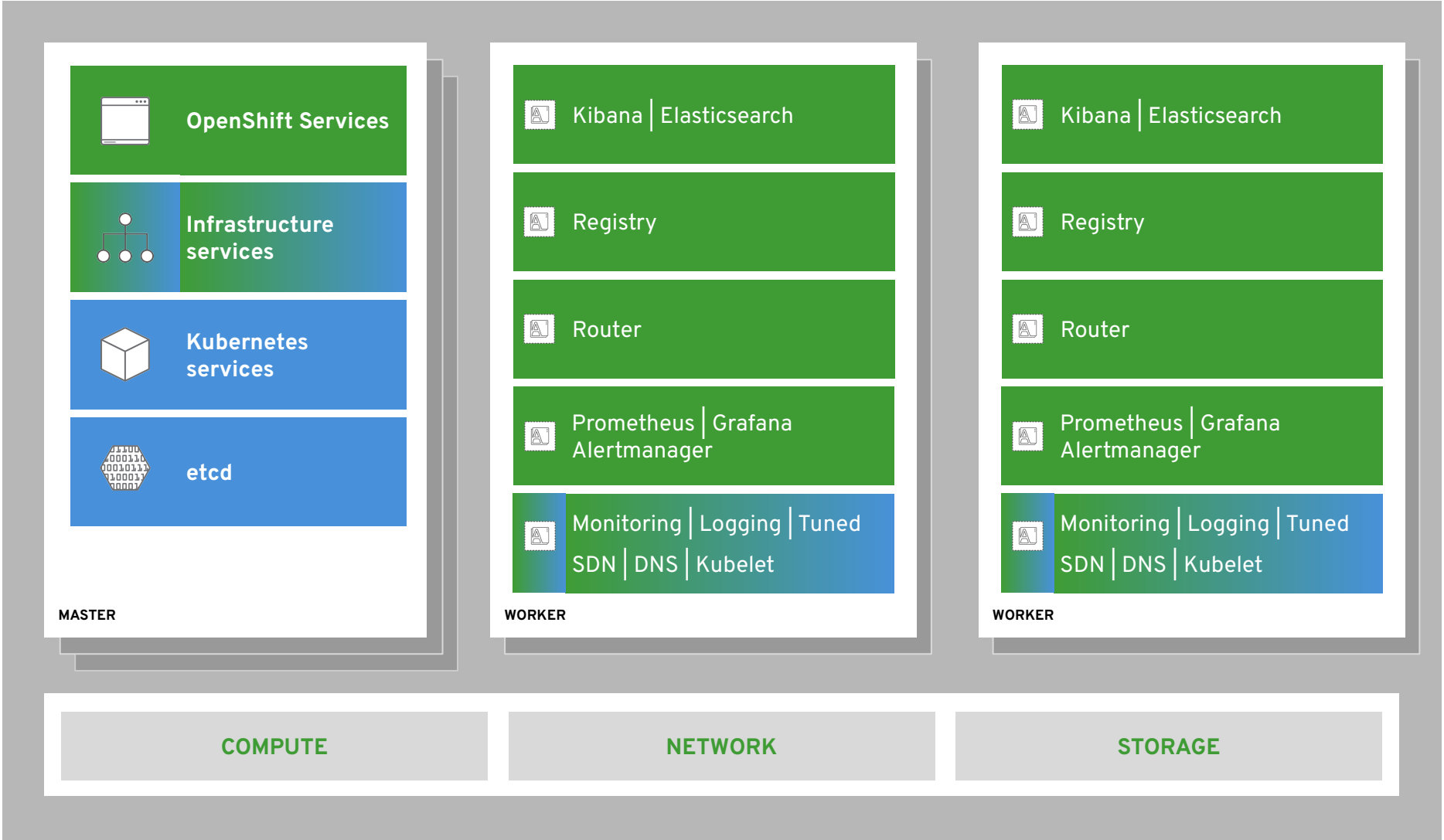
OpenShift のアーキテクチャ - Cluster モニタリング



OpenShift のアーキテクチャ - ロギング機能



OpenShift のアーキテクチャ - 外部連携

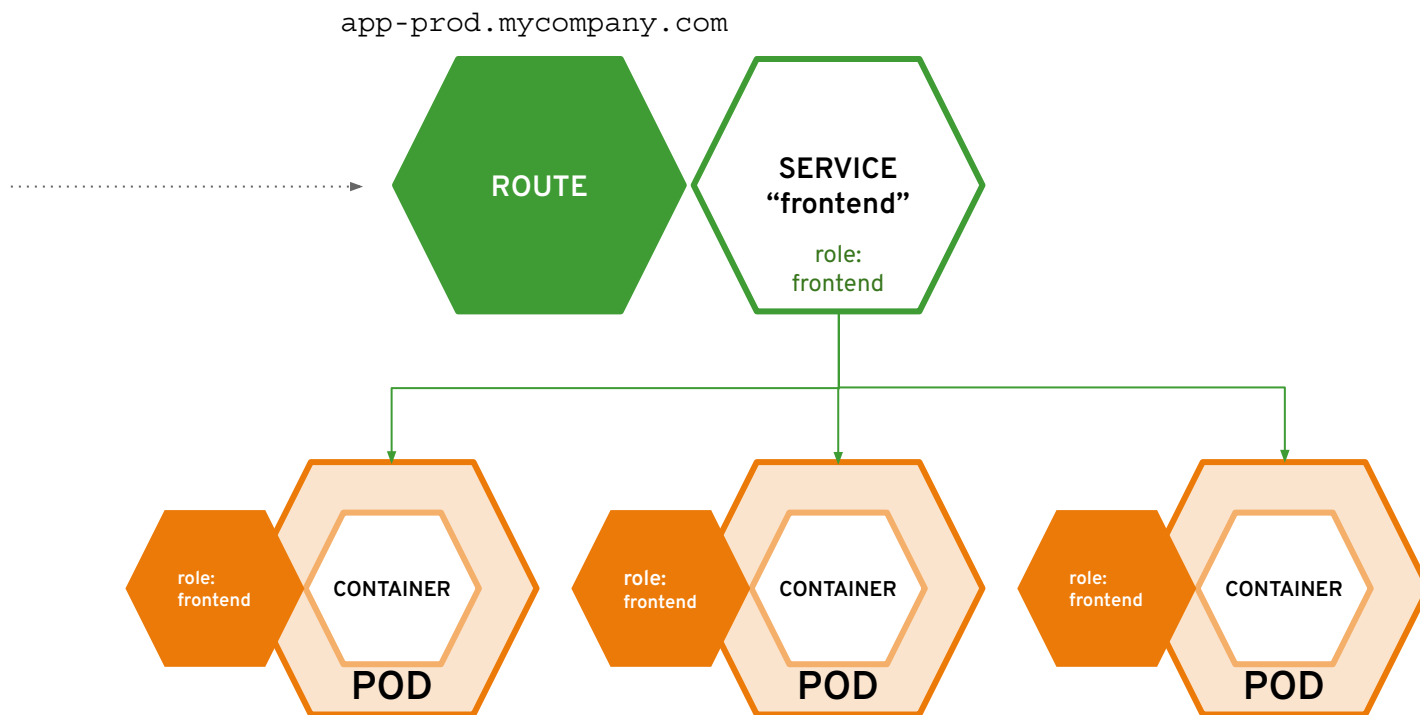
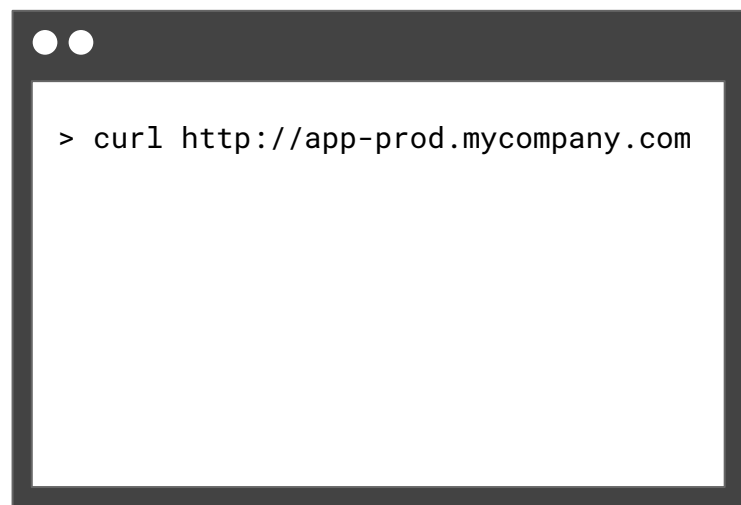


Note:

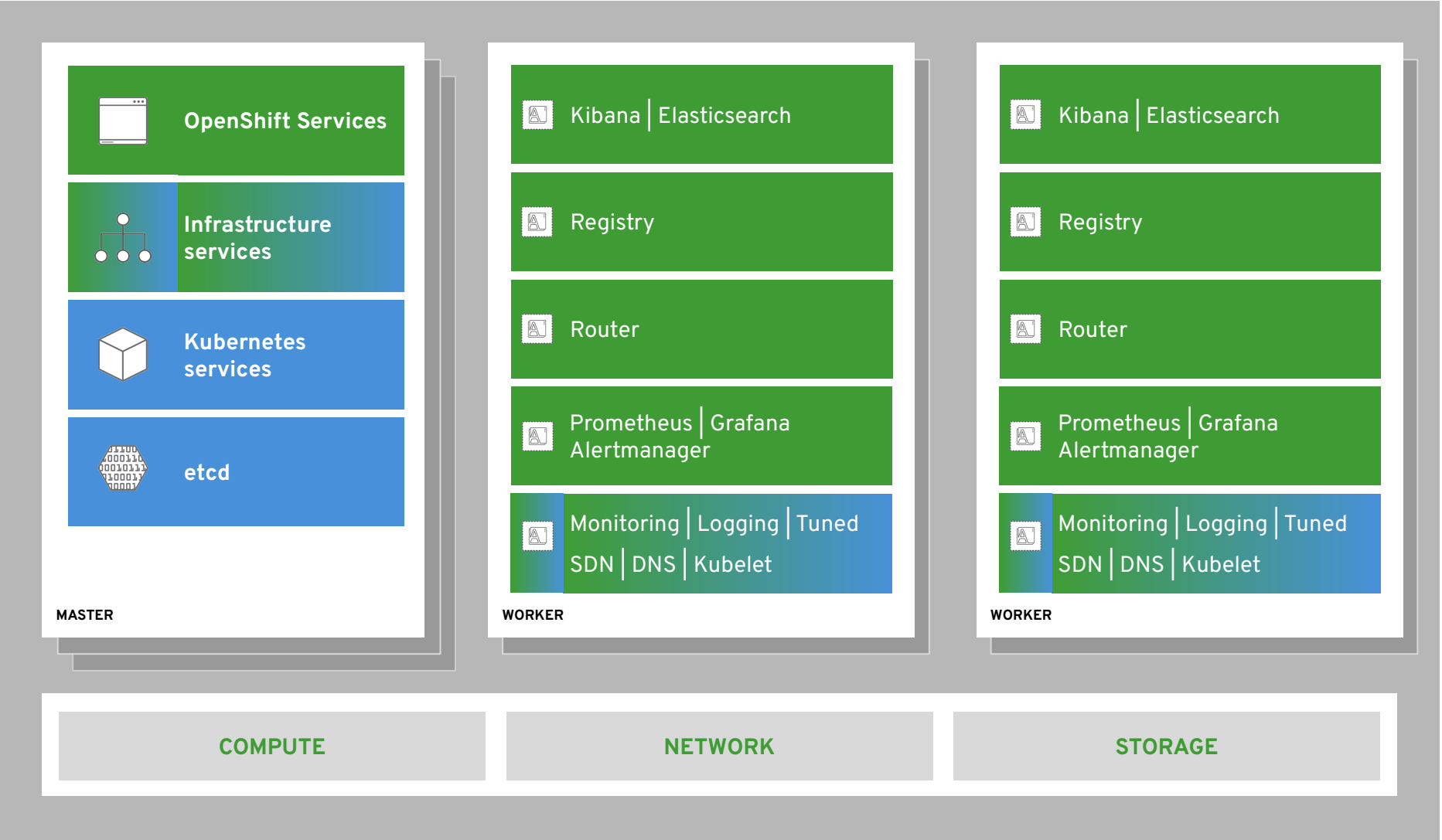
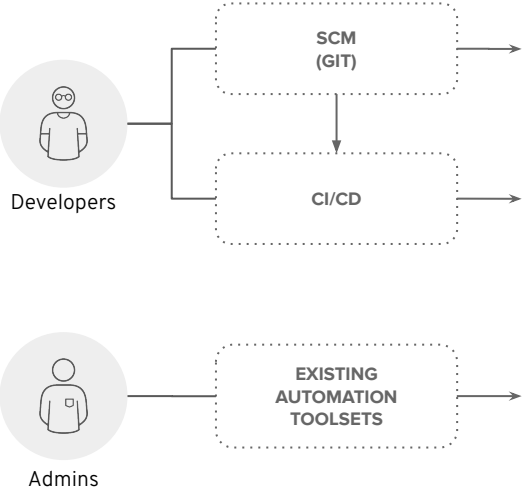
Router

Router (コントローラー)

環境外のクライアントから、外部 URL としてサービスにアクセスが可能



RouterはA/B DeploymentやTLS終端など高度な機能を利用することができます。
Kubernetesの標準オブジェクトであるIngressに相当。

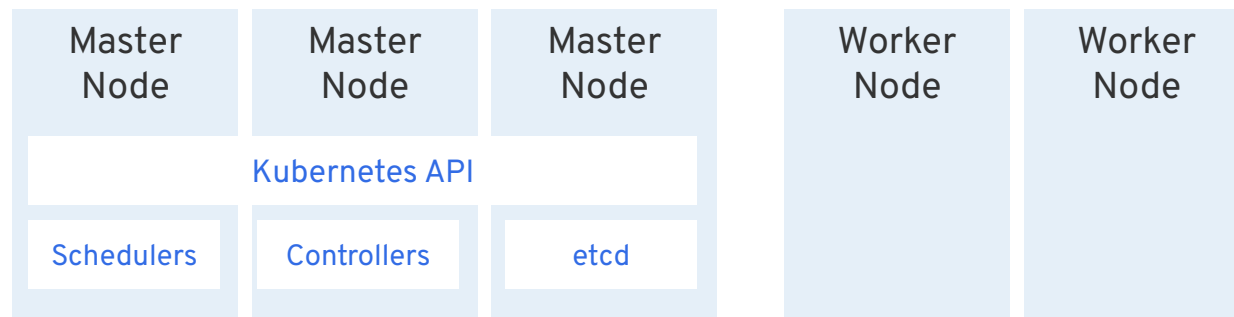


Kubernetes 機能

OpenShift 機能

OpenShiftクラスタのインストールの検証

ワークショップの環境は、AWS上にIPI (Installer-provisioned infrastructure) インストールされています



Master Node (3 nodes)

Master Nodesには、コンテナを制御するコンポーネントと、クラスタの状態を構成管理するデータ(etcd)があります。

Controllers: リソースのデプロイを行う機能

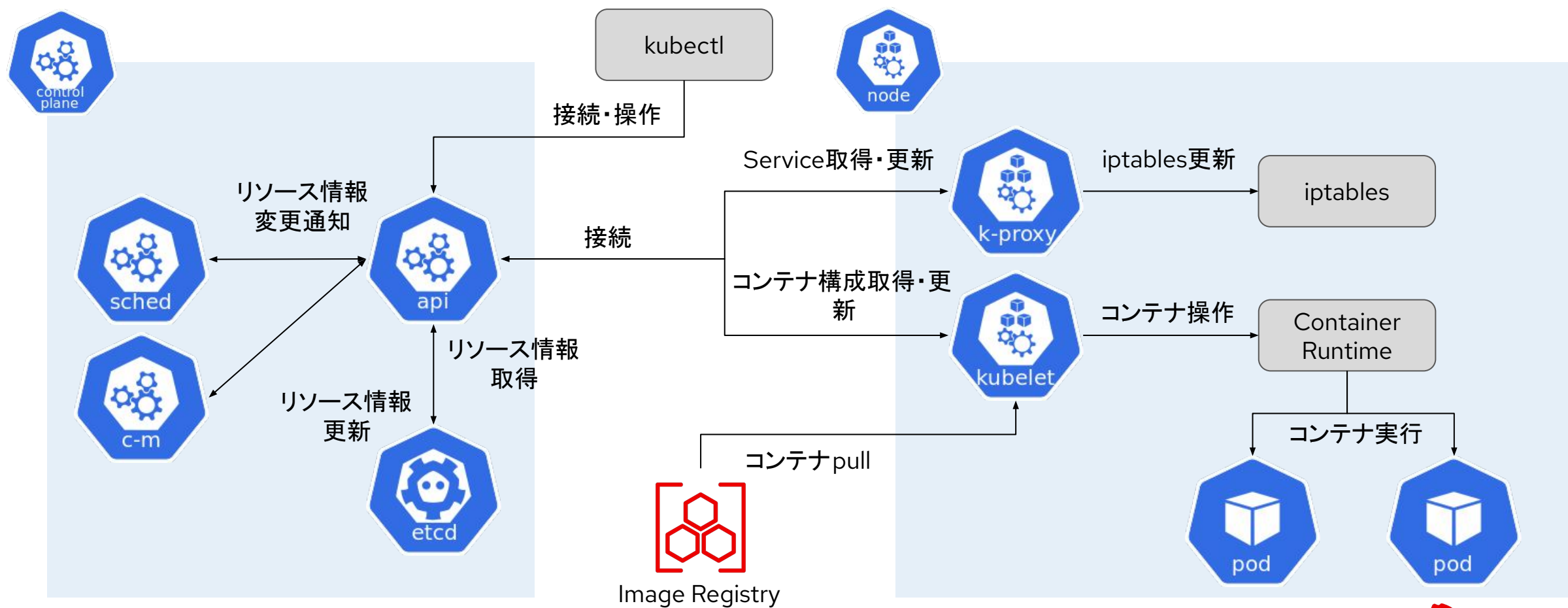
Schedulers: リソースの空き状況を管理する機能

Worker Node (2~N nodes)

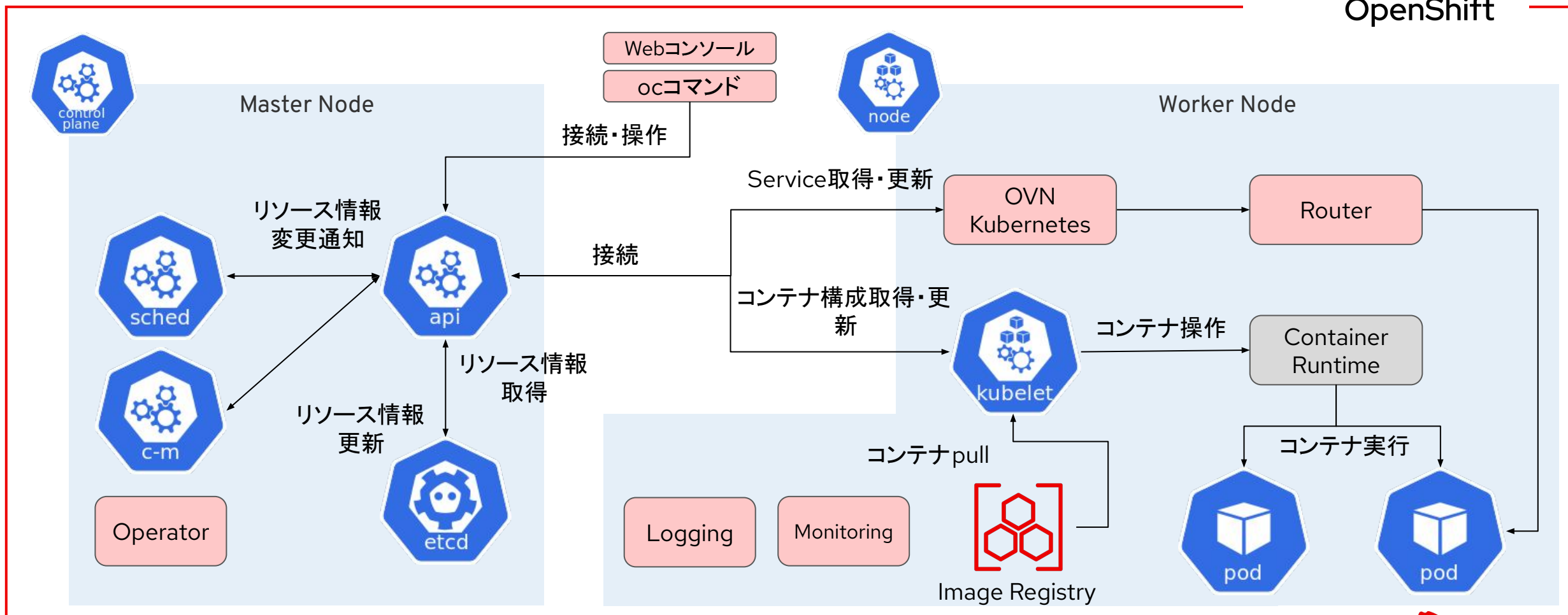
Worker Nodesでは、スケジュールされたコンテナがデプロイされ、コンテナを死活監視します。



Kubernetesのアーキテクチャ



OpenShiftのアーキテクチャ

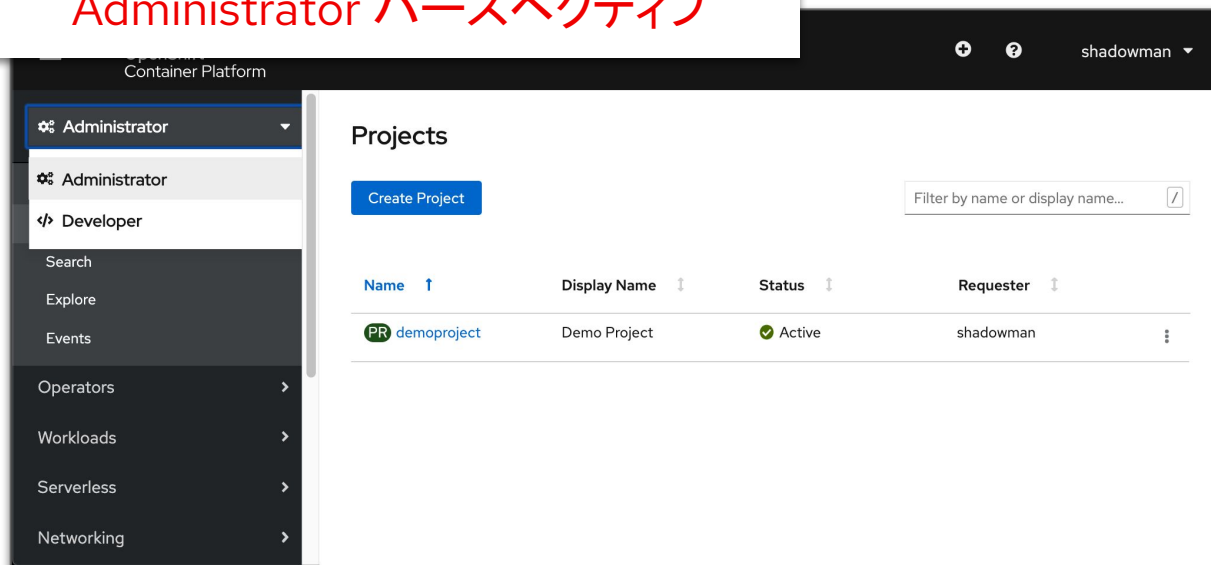


OpenShift Web コンソール

2つのパースペクティブ: Administrator / Developer

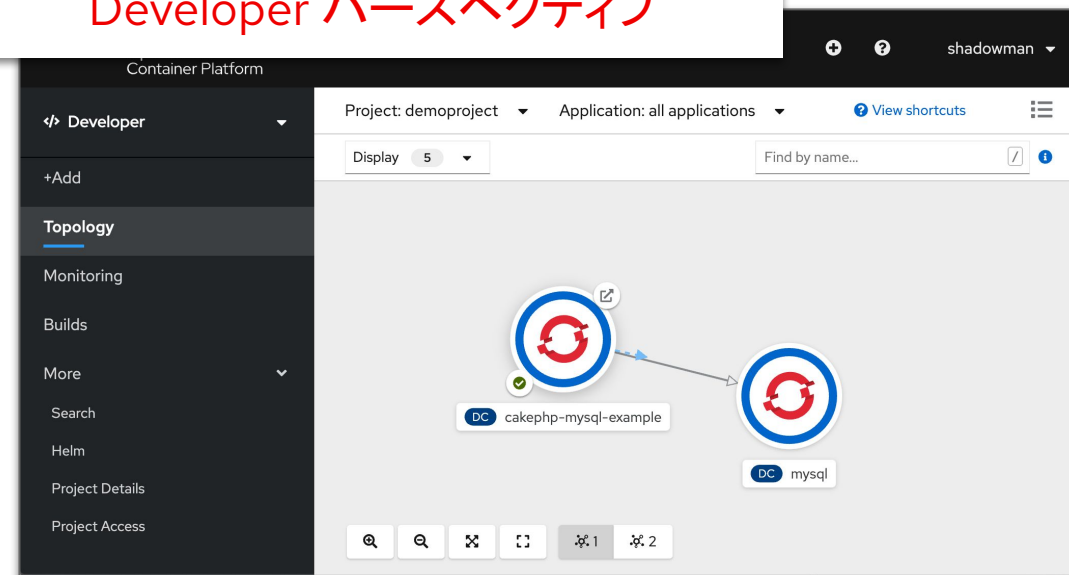
OpenShift Webコンソールは、Podとして実行される。

Administrator パースペクティブ



- ステータス、アクティビティ/イベント、インベントリ、キャパシティ、使用状況
- 標準リソースタイプはすべて管理可能
- ロギングとメトリック
- その他確認できるもの: クラスターの更新、オペレーター、CRD、ロールバインディング、リソースクォータ

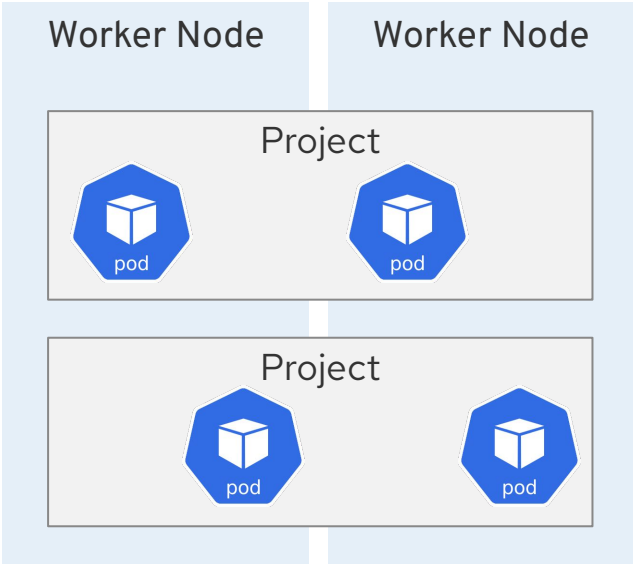
Developer パースペクティブ



- トポロジービュー
- アプリケーション中心
- コンポーネントとステータス、ルート、ソースコードを表示
- 矢印を使った関係の構築
- コンポーネントをアプリケーションに簡単に追加

アプリケーション管理の基礎

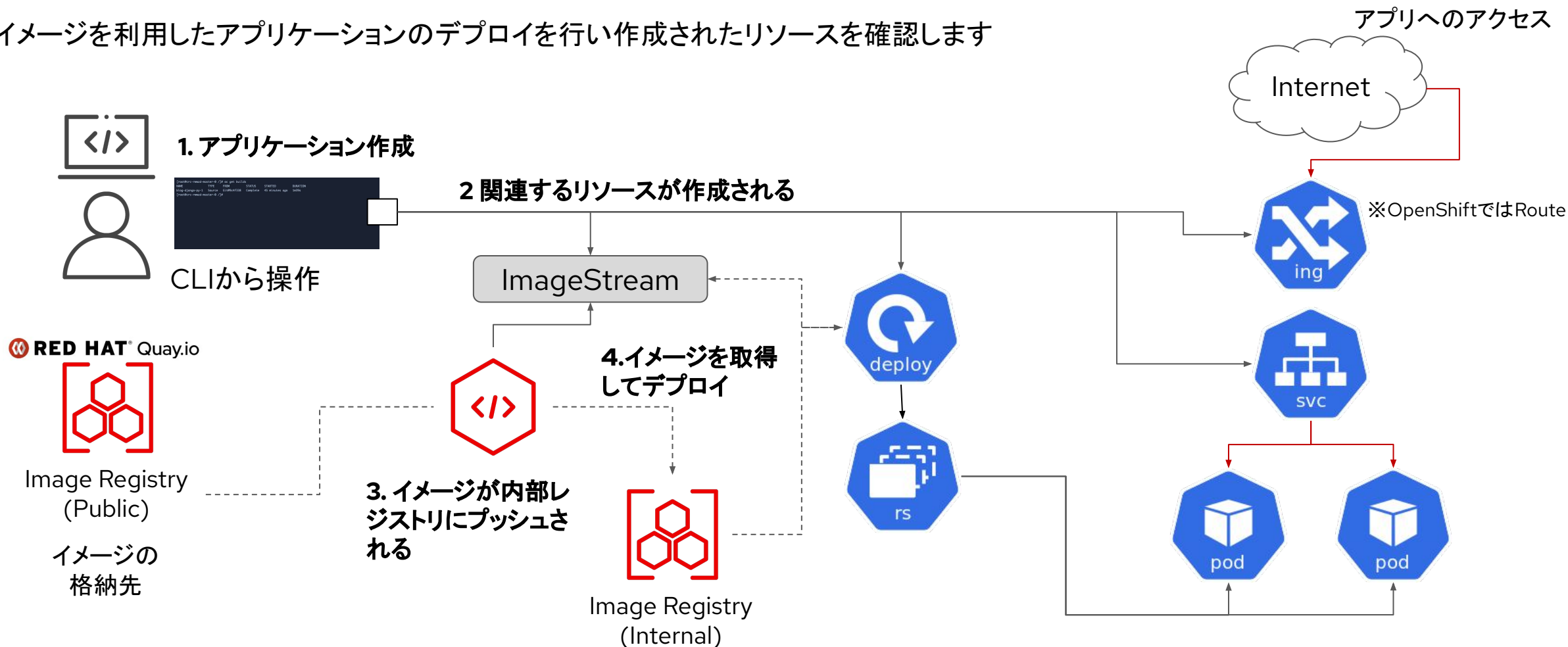
OpenShiftのコアとなる概念について学びます



リソース	概要
Project	Project単位でリソースを分離。管理の観点からは、各Projectはテナントのように考えることができます
Pod	ホスト上に一緒にデプロイされた1つまたは複数のコンテナ。PodはOpenShiftで定義、デプロイ、管理できるコンピュータリソースの最小の単位です
Service	OpenShift環境内でPodのようなグループを見つけるための抽象化レイヤーを提供。OpenShift環境内からPodにアクセスする必要があるものとの間の内部プロキシ/ロードバランサーとしても機能します
Deployment	OpenShift内に何をどのようにデプロイするかを定義します
ReplicaSet	必要な数のPodを確実に存在させるために使用されます。ReplicaSetはOpenShiftが自己修復する方法を提供します
Route	OpenShift外のクライアントがOpenShift内で実行されているアプリケーションにアクセスする方法を提供します

アプリケーション管理の基礎

イメージを利用したアプリケーションのデプロイを行い作成されたリソースを確認します



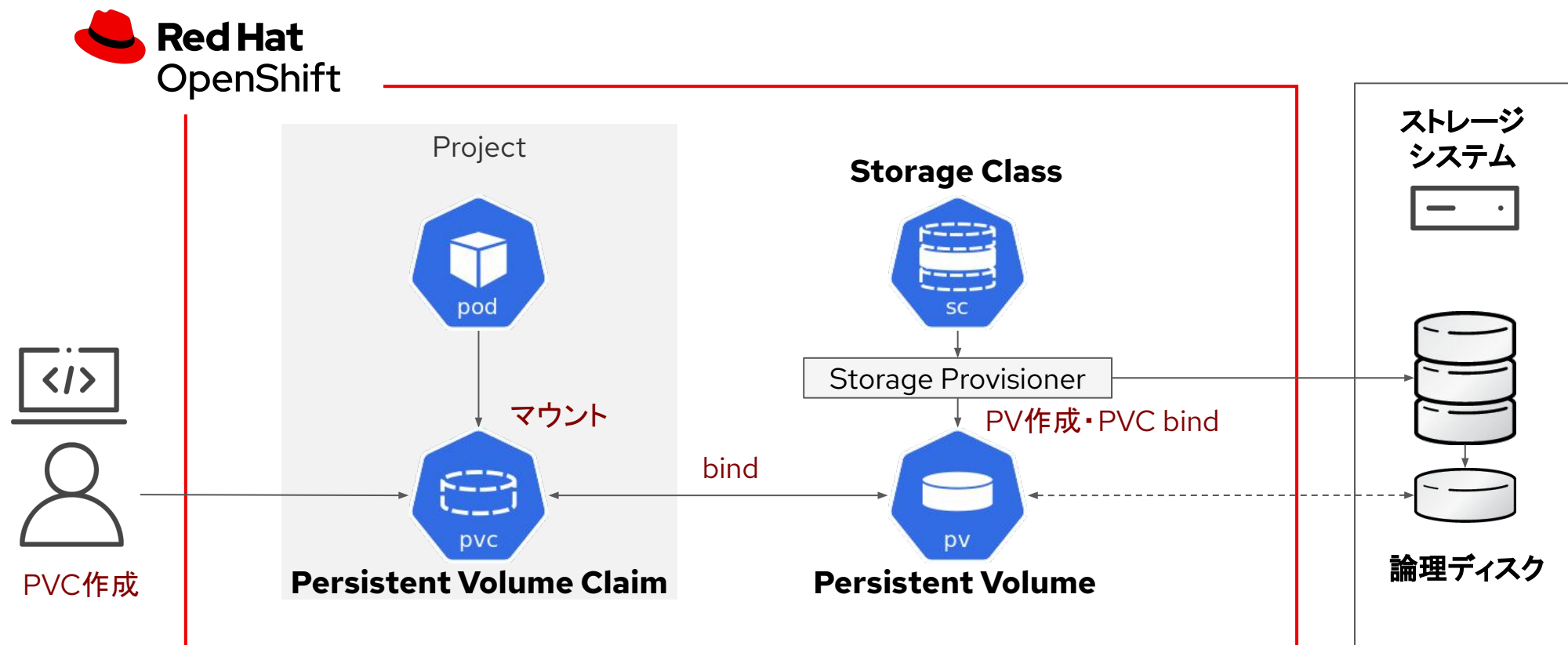
アプリケーション管理の基礎

イメージを利用したアプリケーションのデプロイを行い作成されたリソースを確認します

1. プロジェクト作成: `oc new-project app-management`
2. アプリケーション作成: `oc new-app quay.io/openshiftroadshow/mapit`
3. アプリケーションに関連するリソースが作成されます
各リソースの内容を確認します
4. routeを作成し、外部からアクセスできることを確認します

アプリケーションストレージの基礎

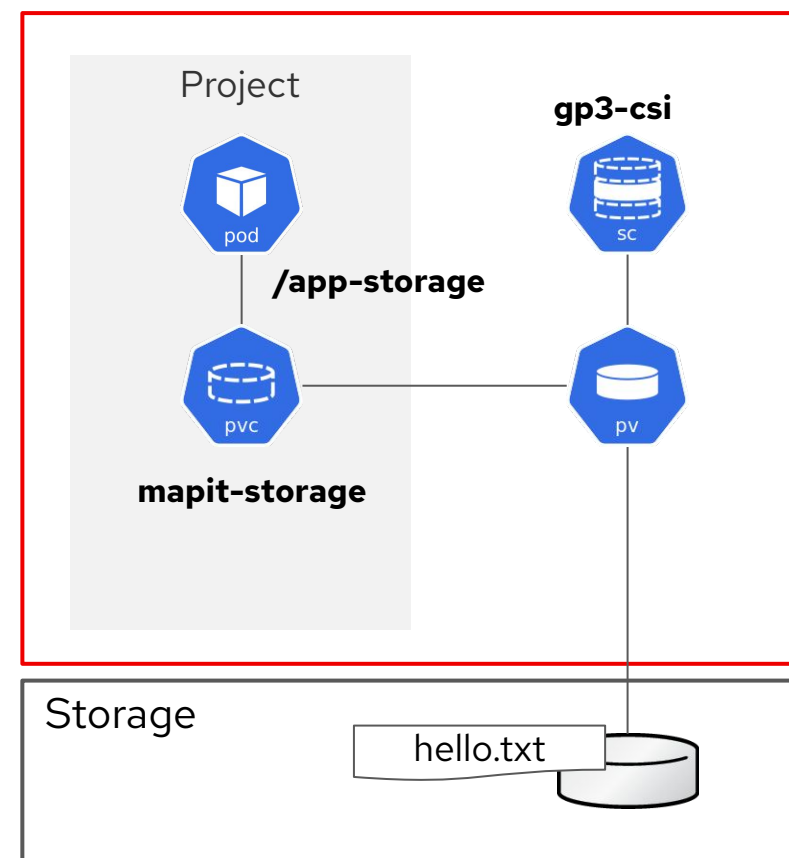
OpenShiftにおける永続ボリューム (Persistence Volume) の動作を確認します



アプリケーションストレージの基礎

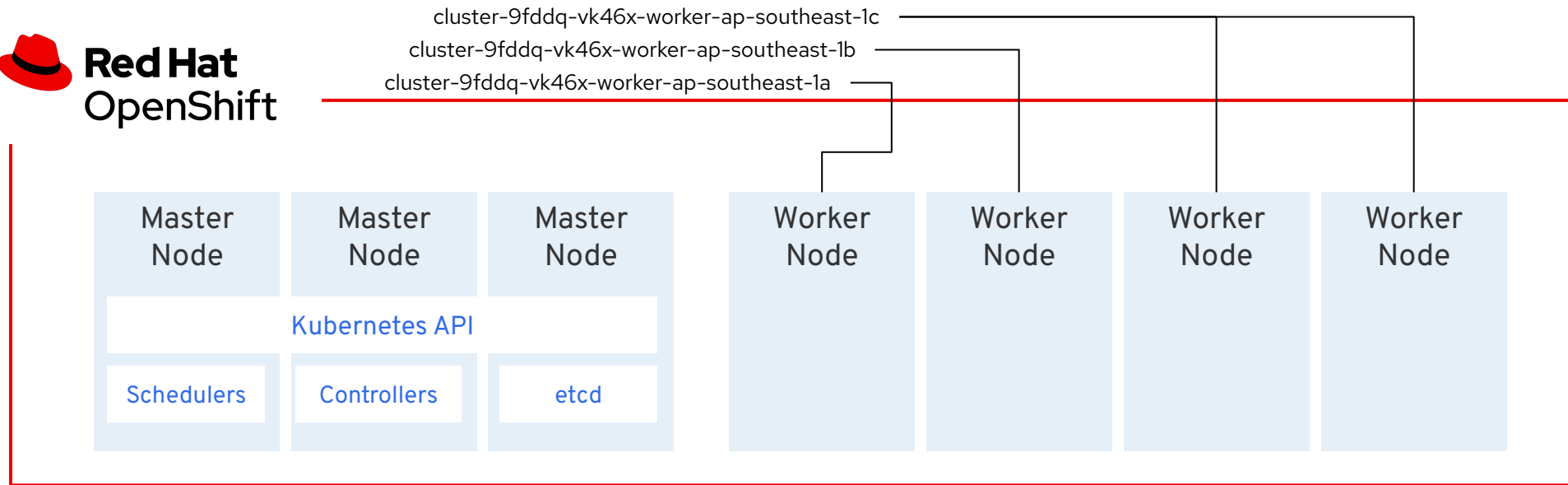
OpenShiftにおける永続ボリューム (Persistence Volume) の動作を確認します

1. アプリケーション mapit で下記の内容の永続ボリュームを使用します
PVC名 : mapit-storage
ReadWriteOnceモード
サイズ : 1Gi
mount-path : /app-storage
2. 永続ボリュームにファイルを作成します
3. 稼働している Pod を削除します
4. 新しい Pod がすぐに稼働します
5. 永続ボリュームに作成したファイルを継続して利用できることを確認します

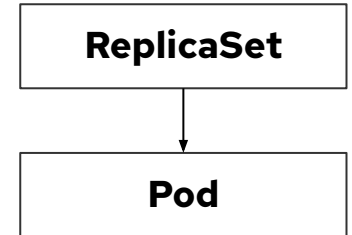


MachineSet, Machine, and Node

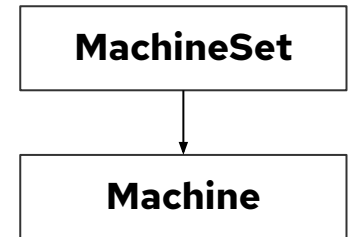
MachineSet/Machineを利用して、ノードを環境に追加(削除)します



ReplicaSetがPodの数をコントロール



MachineSetがMachine(node)の数をコントロール



MachineSet

cluster-9fddq-vk46x-worker-ap-southeast-1a	1	1	1	1	31h
cluster-9fddq-vk46x-worker-ap-southeast-1b	1	1	1	1	31h
cluster-9fddq-vk46x-worker-ap-southeast-1c	0	0			31h

Machine

cluster-9fddq-vk46x-worker-ap-southeast-1a-hpwng	Running
cluster-9fddq-vk46x-worker-ap-southeast-1b-d7dll	Running

cluster-9fddq-vk46x-worker-ap-southeast-1a	1	1	1	1	32h
cluster-9fddq-vk46x-worker-ap-southeast-1b	1	1	1	1	32h
cluster-9fddq-vk46x-worker-ap-southeast-1c	2	2	2	2	32h

cluster-9fddq-vk46x-worker-ap-southeast-1a-hpwng	Running
cluster-9fddq-vk46x-worker-ap-southeast-1b-d7dll	Running
cluster-9fddq-vk46x-worker-ap-southeast-1c-l5jjw	Provisioned
cluster-9fddq-vk46x-worker-ap-southeast-1c-zm2dt	Provisioned

Node

ip-10-0-198-39.ap-southeast-1.compute.internal
ip-10-0-209-58.ap-southeast-1.compute.internal

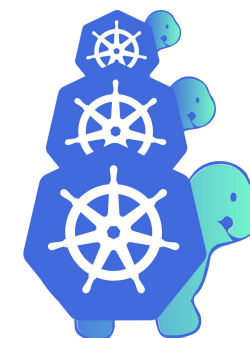
Machine-API

Machine-APIはアップストリームのKubernetes Cluster APIをOpenShiftに機能実装したもの

Cluster API

kubernetesクラスタのプロビジョニング、アップグレード、運用を簡素化するための宣言型 APIとツールの提供にフォーカス
クラスタのライフサイクル管理を自動化する
多種多様なインフラストラクチャ環境において、一貫性のある再現可能なクラスタのデプロイを可能にする

- ・ノードのスケールイン・スケールアウトがコマンドラインから可能
- ・ノードのスケール調整そのものを自動化 (autoscale)
- ・ノードのローリングアップデート



参考資料: <https://www.slideshare.net/ssuser804f1a/machine-configoperator>

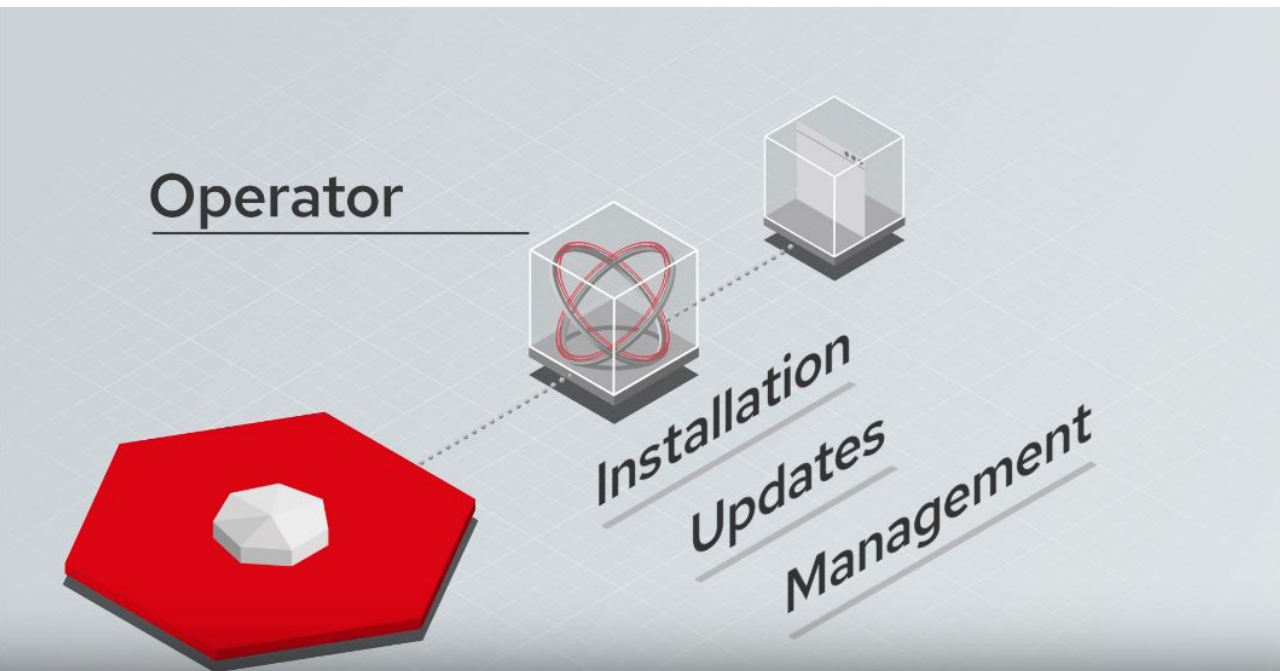
Operator とは

運用の知見をコード化し、アプリケーションの運用を自動化する

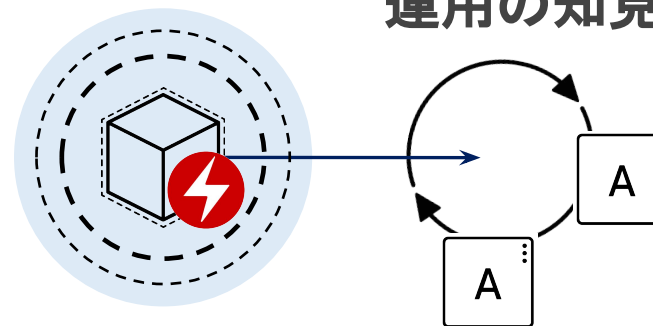
Kubernetes / OpenShift 上のアプリケーション運用における運用の知見をコード化し、パッケージ化したもの

アプリケーション運用に必要な以下のような作業を自動的に行う

- ▶ インストール
- ▶ リソーススケーリング
- ▶ バックアップ
- ▶ アップデート

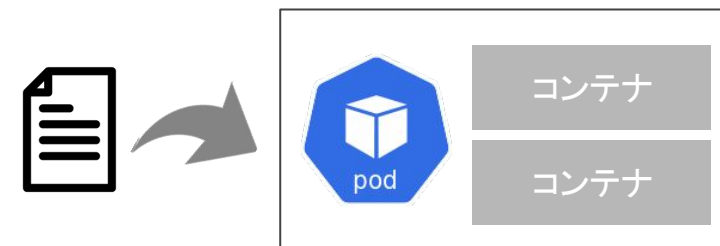


運用の知見をコード化



Operator を使うとカスタムリソースが使える

- ▶ Kubernetesに対して新しいリソースを追加する(APIの拡張)
 - ・ Custom Resource Definition(CRD)に定義を記載
 - ・ カスタムリソースを利用することができる
 - Kubernetes のデフォルトのリソース以外も、Kubernetes 内で利用ができる
 - Kubernetes 自動化機能のメリットを活用可能



リソース(K8s 組み込み)

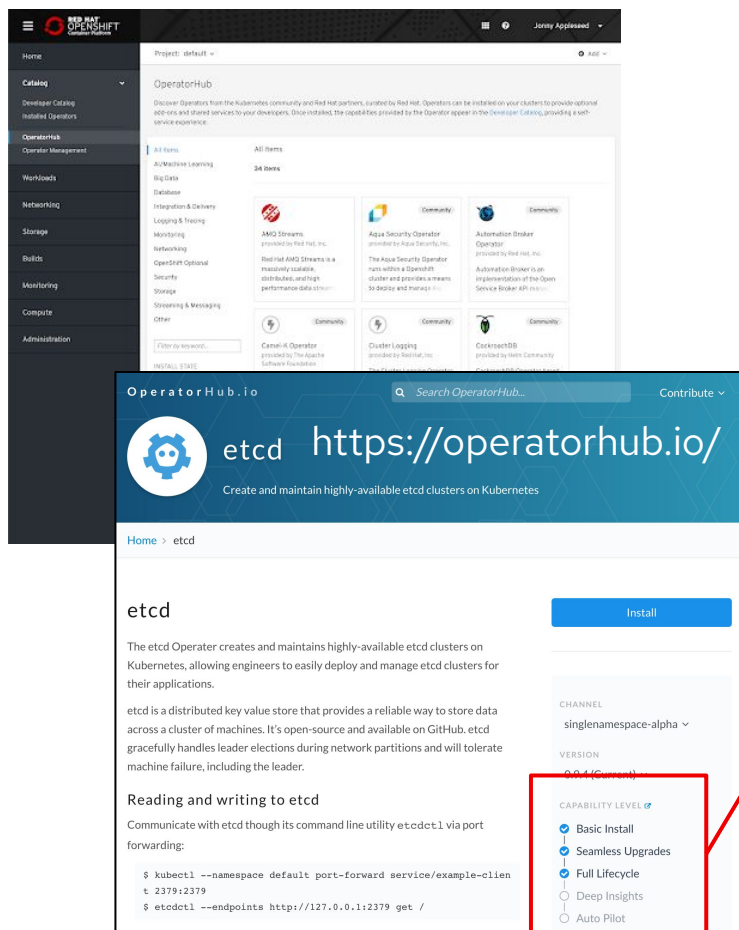
StatefulSets
Pods
Secrets
DaemonSet
Job
CronJob
:

カスタム リソース(例)

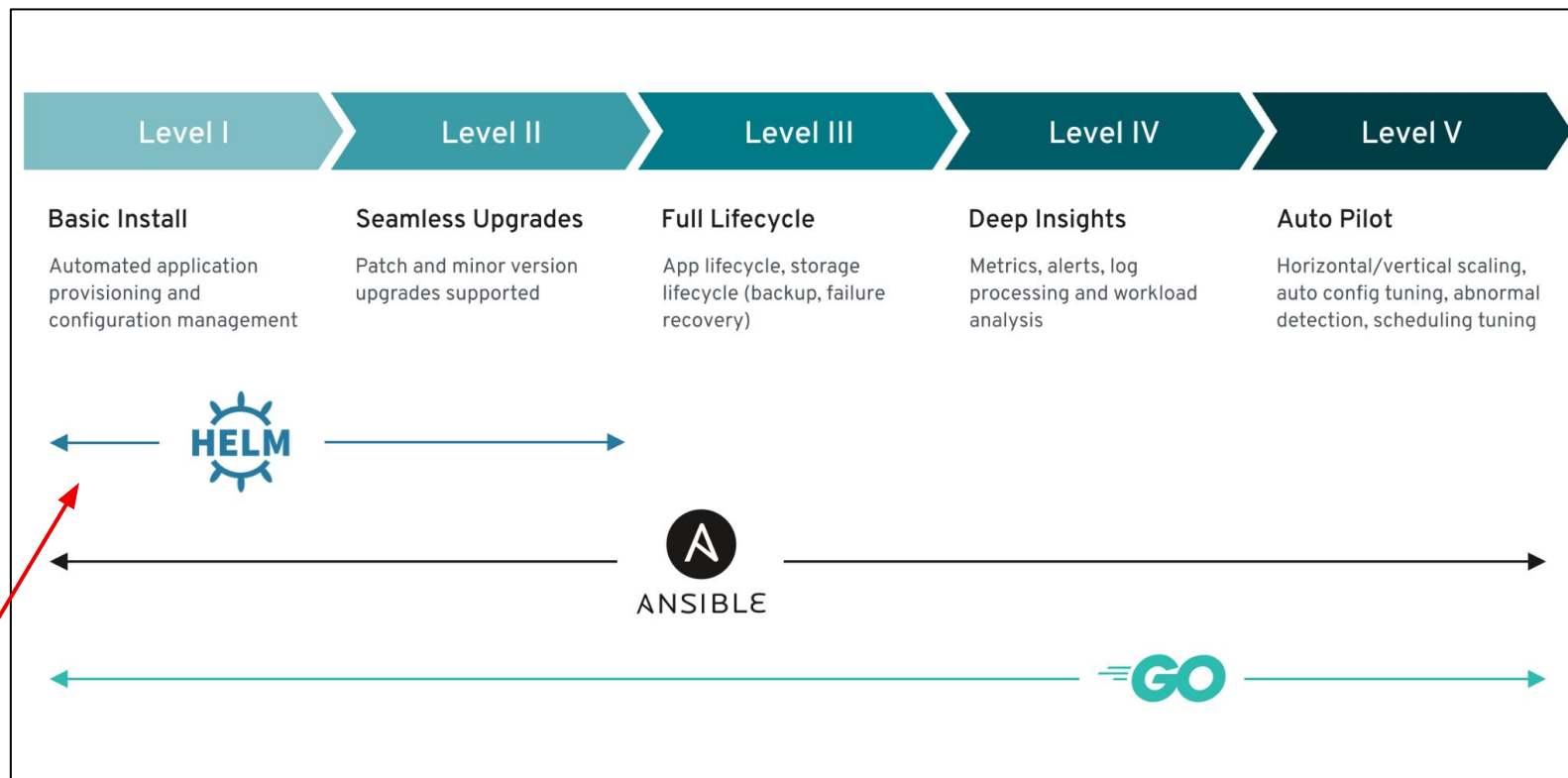
Kafka
KafkaTopic
KafkaUser
KafkaConnect
KafkaConnector
:

リソースが
新しく
定義できる

Operator HubとOperatorの成熟度モデル

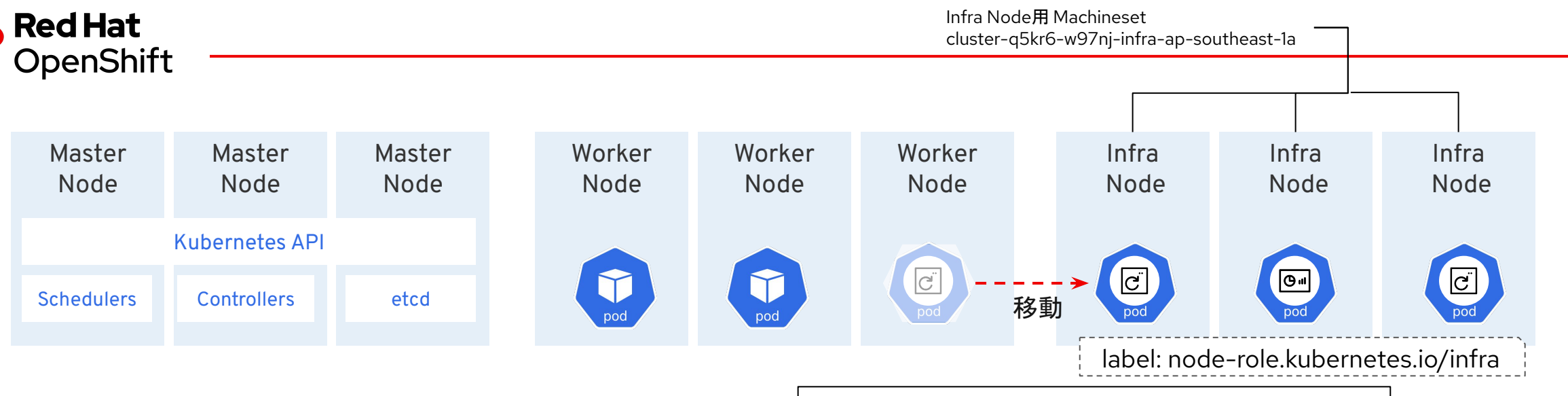


K8s Operatorは提供するアプリケーションまたはワークロードのライフサイクル管理機能に関して、さまざまな成熟度を持っています。この機能モデルは、ユーザーが Operatorに期待できる機能レベルを表現しています。



インフラストラクチャノード と Operator

Infra Nodeを作成し、Worker Node上で稼働しているPod (Router, Registry等) をInfra Nodeに移動させます



Infra Nodeには、OpenShiftの追加サービスを管理するためのコンポーネントが配置されます

Infra Nodeに配置できる機能

- Router
- OpenShift-included Registry
- OpenShift cluster monitoring
- OpenShift log aggregation
- など

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  finalizers:
    - ingresscontroller.operator.openshift.io/finalize
  name: default
  namespace: openshift-ingress-operator
spec:
  nodePlacement:
    nodeSelector:
      matchLabels:
        node-role.kubernetes.io/infra: ""
```

例) Router PodがInfra Nodeで稼働するようnodeSelectorを指定



インフラストラクチャノード と Operator

例) cluster-5gqmq-pkmg1-worker-ap-southeast-1a

spec:

replicas: 1

selector:

matchLabels:

machine.openshift.io/cluster-api-cluster: cluster-5gqmq-pkmg1

machine.openshift.io/cluster-api-machineset: cluster-5gqmq-pkmg1-worker-ap-southeast-1a

template:

metadata:

labels:

machine.openshift.io/cluster-api-cluster: cluster-5gqmq-pkmg1

machine.openshift.io/cluster-api-machine-role: worker

machine.openshift.io/cluster-api-machine-type: worker

machine.openshift.io/cluster-api-machineset: cluster-5gqmq-pkmg1-worker-ap-southeast-1a

spec:

lifecycleHooks: {}

metadata: {}

providerSpec:

value:

ami:

id: ami-0f827a1be73b9de83 AmazonマシンイメージID

レプリカ数

クラスタID
machineset名

クラスタID

machineset名

インフラストラクチャノード と Operator

例) cluster-5gqmq-pkmg1-worker-ap-southeast-1aをもとにInfra Node用の新しいmachinesetを作成しスケールアウト

machineset-generator.shの内容

```
#!/bin/bash
export AMI=$(oc get machineset -n openshift-machine-api -l 'machine.openshift.io/os-id!=Windows' -o
jsonpath='{.items[0].spec.template.spec.providerSpec.value.ami.id}')
export CLUSTERID=$(oc get infrastructures.config.openshift.io cluster -o jsonpath='{.status.infrastructureName}')
export REGION=$(oc get infrastructures.config.openshift.io cluster -o jsonpath='{.status.platformStatus.aws.region}')
export COUNT=${1:-3}
export NAME=${2:-workerocs}
export SCALE=${3:-1}

$( cd "$( dirname "${BASH_SOURCE[0]}" )" && pwd )/machineset-cli -scale $SCALE -name $NAME -count $COUNT
-ami $AMI -clusterID $CLUSTERID -region $REGION
```

AMI情報

クラスタID

リージョン情報

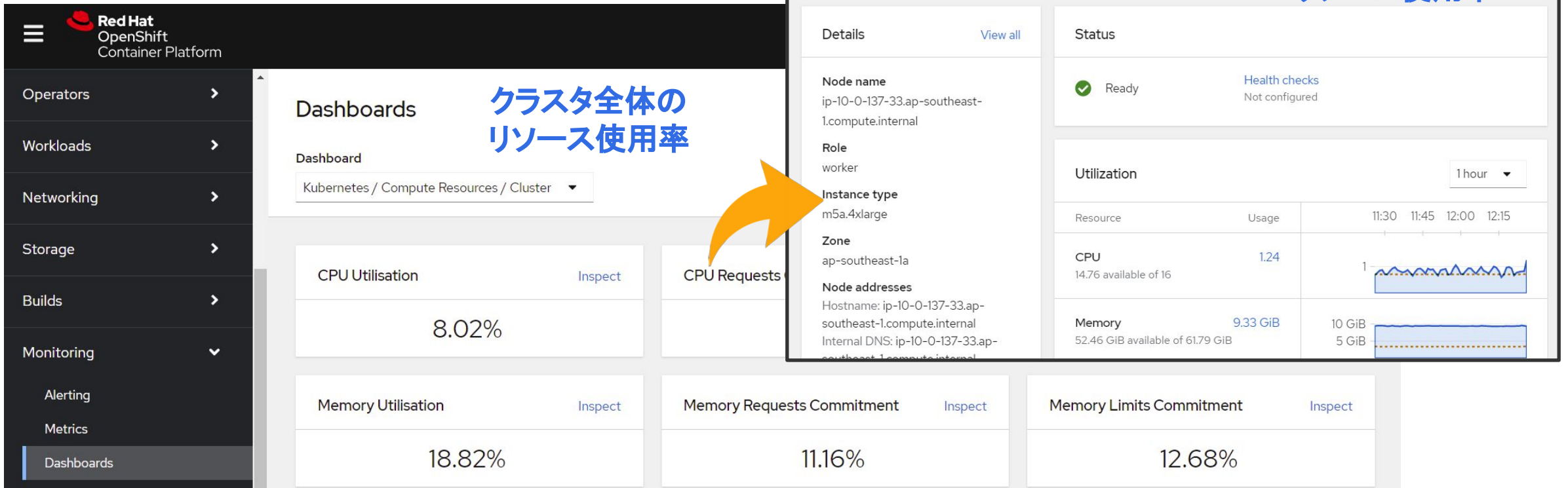
Infra Node用Machineset作成とスケールアウトの実行

```
bash ${HOME_PATH}/support/machineset-generator.sh 1 infra 0 | oc create -f -
export MACHINESET=$(oc get machineset -n openshift-machine-api -l machine.openshift.io/cluster-api-machine-role=infra -o
jsonpath='{.items[0].metadata.name}')
oc patch machineset $MACHINESET -n openshift-machine-api --type='json' -p='[{"op": "add", "path":
"/spec/template/spec/metadata/labels", "value": {"node-role.kubernetes.io/worker": "", "node-role.kubernetes.io/infra": ""}}]'
oc scale machineset $MACHINESET -n openshift-machine-api --replicas=3
```

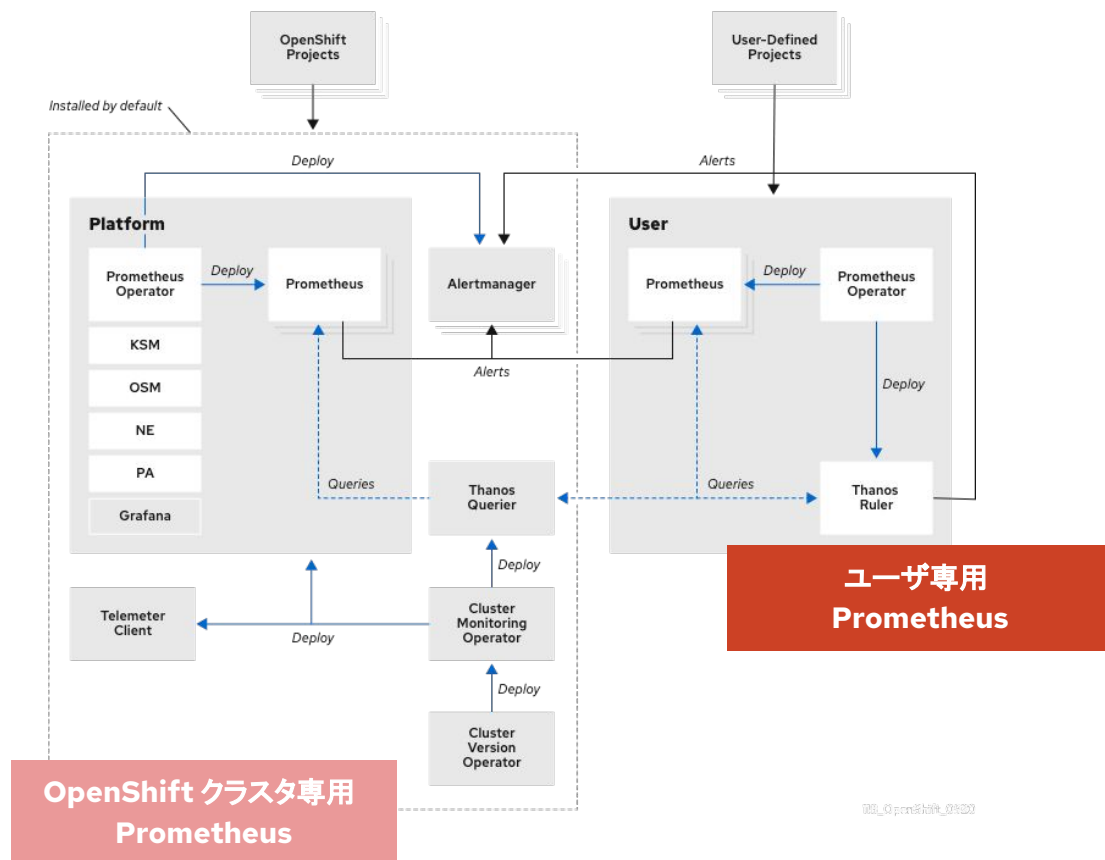
Cluster Monitoring

OpenShiftをインストールした時点で、クラスタに関する監視はCluster Monitoring(Prometheus)によって設定済みです。(アラート含む)

初期設定や更新作業は自動で設定されます。



Cluster Monitoring の構成



- OpenShift クラスタ専用 Prometheus [1]
 - 対象
 - Kubernetes/OpenShift API、etcd、CoreDNS、ホストOS周り
 - 備考
 - Cluster Monitoring に設定されているルールのカスタマイズはサポート対象外(監視対象は、OpenShiftに委ねられている)

・K8sオブジェクトのメトリクス収集:
例: Pod数、Podのリソース利用量、Serviceのラベル、etc

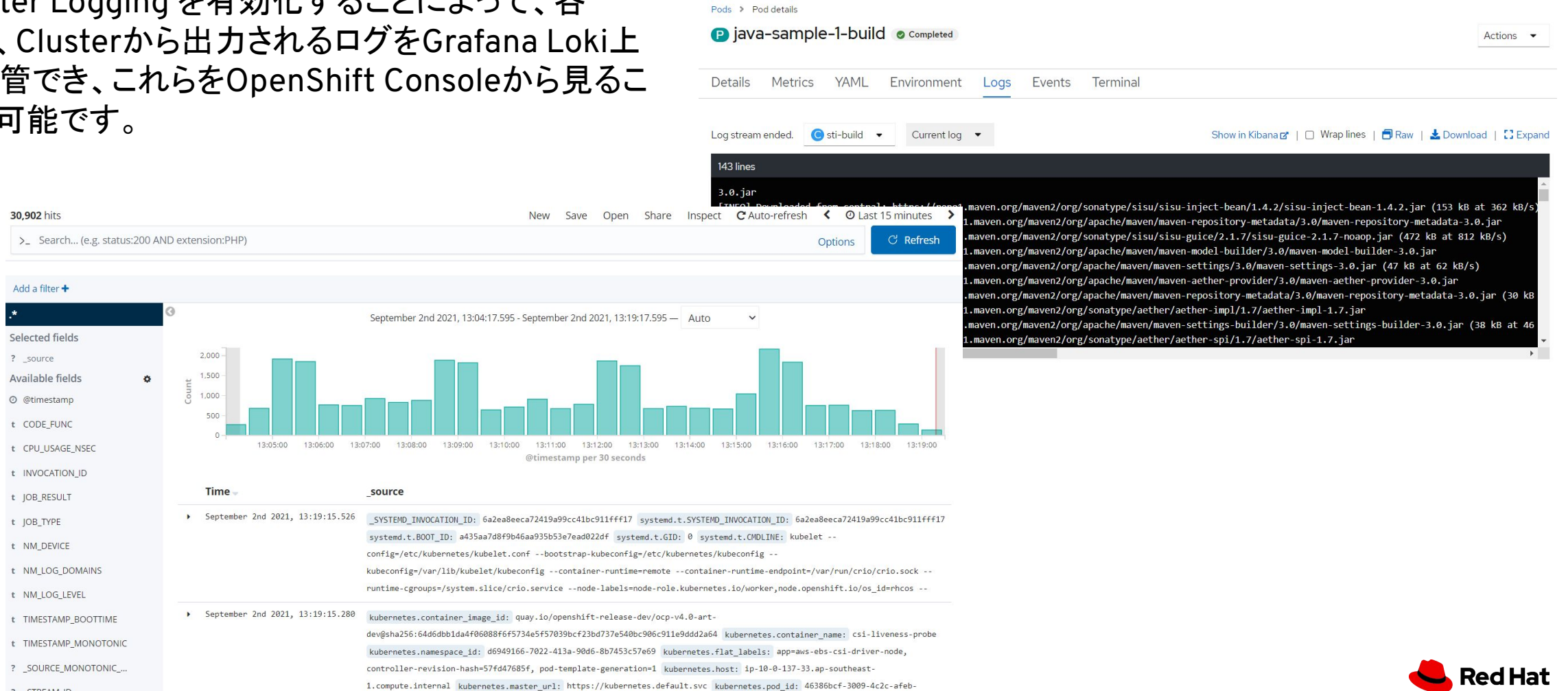
・K8sノードのメトリクス収集:
例: NodeのCPU、Memory、I/O、etc

- ユーザ専用 Prometheus
 - Apache、Nginxなどユーザがデプロイしたアプリケーションの固有のメトリクス収集

・アプリケーション固有のメトリクス収集:
例: HTTPセッション数、リクエストレスポンスタイム、データ容量 etc

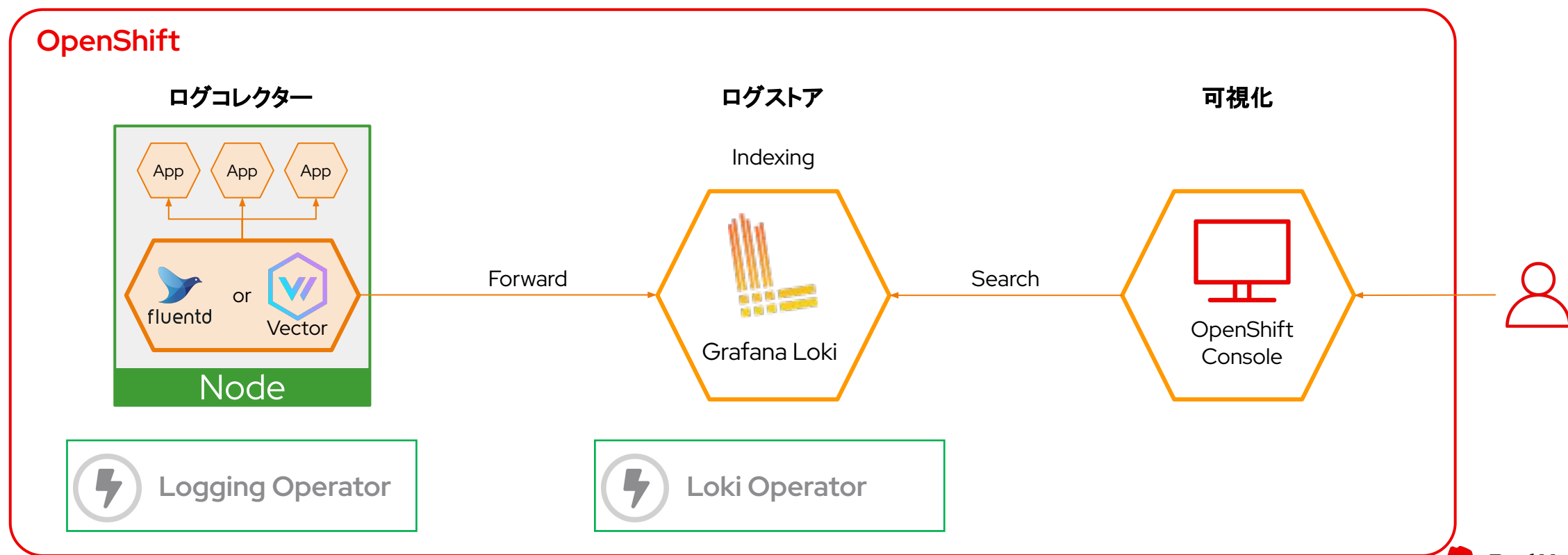
Cluster Logging

Cluster Logging を有効化することによって、各 Pod、Clusterから出力されるログをGrafana Loki上に保管でき、これらをOpenShift Consoleから見る事が可能です。



OpenShiftクラスタロギングスタック

OpenShiftのロギングスタックはLogging OperatorとLoki Operatorを使って設定、運用自律化を行います。
各ノード上のPodやノード内のauditのログを収集する**ログコレクター**、ログにIndexを付与して保管する**ログストア**で構成され、OpenShiftのコンソールから閲覧・検索ができます。



Note:

アプリケーションの監視設定もOpenShiftで簡素化

k8sでJavaを見守る新常識

伊藤ちひろ
Chihiro Ito

監視についてよくある問題

- 監視なんて不要だと思って監視の仕組みを作成するコストはない
- 監視が必要だと思っているが、監視についての知識がない
- 監視環境の管理なんてしたくない
- 監視をしても問題の分析に必要な情報が取れていない
- 監視しても問題の予兆を検知できず、問題が顕在化する
- 問題を製品のせいにし、別の製品へ移行して再発する

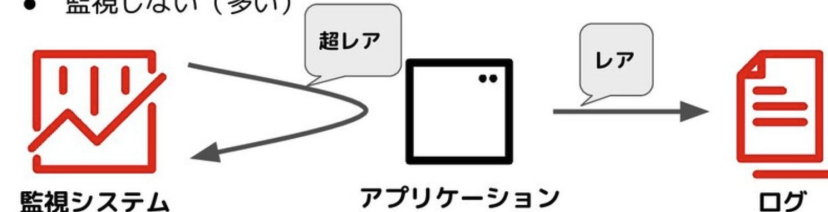
今日の
本題

8



従来の監視方法の例

- 監視システムを構築し、監視対象を登録（超レア）
- 情報をログファイルへ保存（レア）
- 監視しない（多い）



26



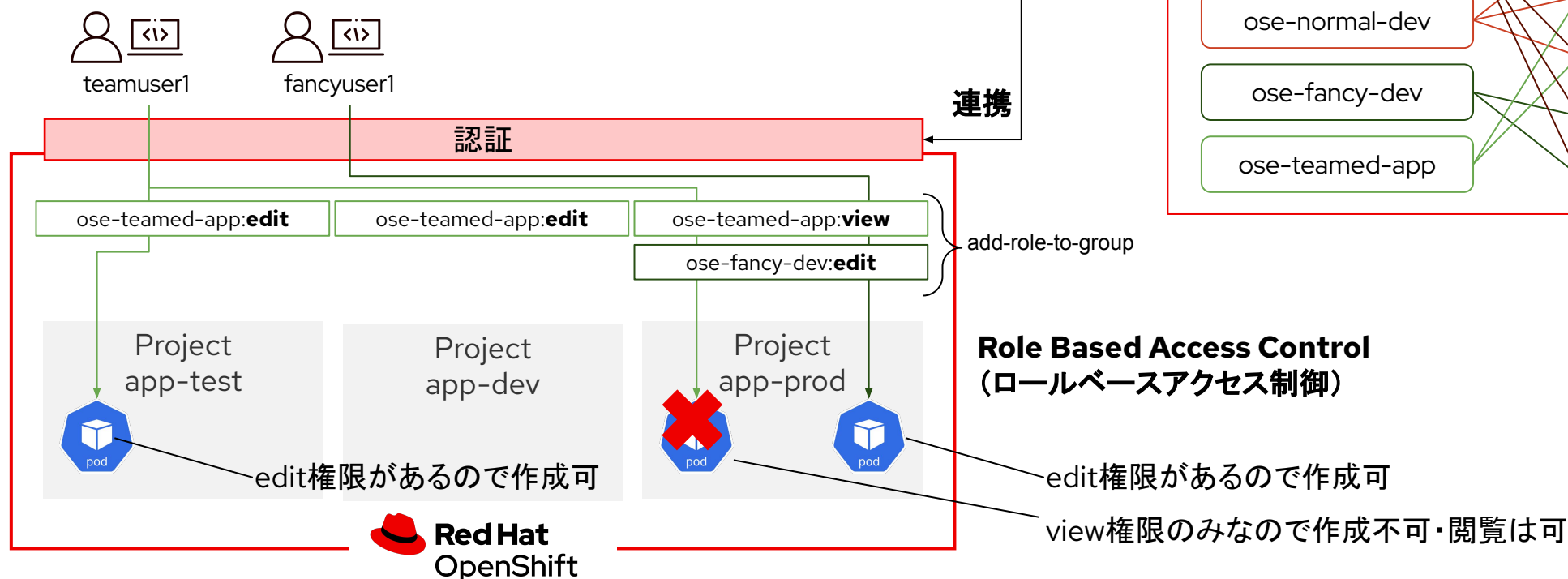
<https://speakerdeck.com/chiroito/k8sdejavawojian-shou-ruxin-chang-shi>



外部認証プロバイダ (LDAP) の設定

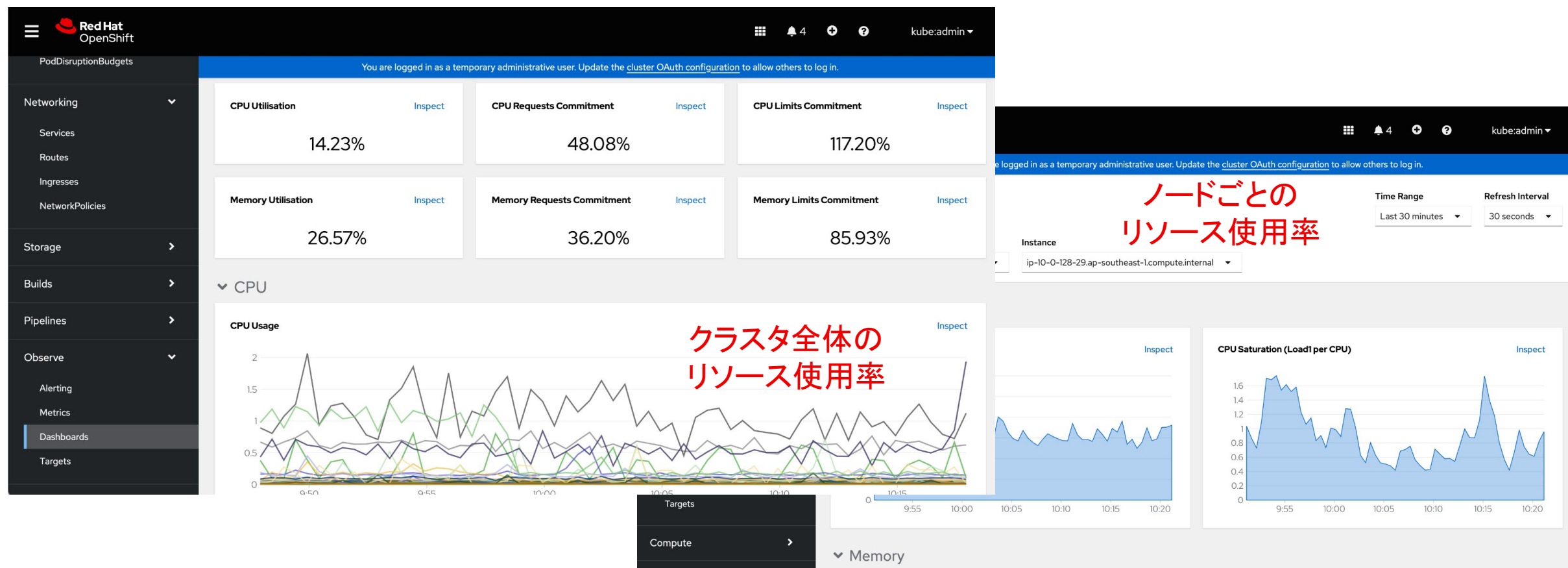
- LDAP連携の設定を行います
- Role Based Access Control設定と動作確認を行います

グループへのクラスターロールの追加: `oc adm policy add-cluster-role-to-group`
 グループへのロールの追加: `oc adm policy add-role-to-group`



OpenShift Monitoring

OpenShiftをインストールした時点で、クラスタに関する監視はCluster Monitoring(Prometheus)によって設定済みです。



プロジェクト・リクエスト・テンプレートとクォータ / 制限

Project / Template

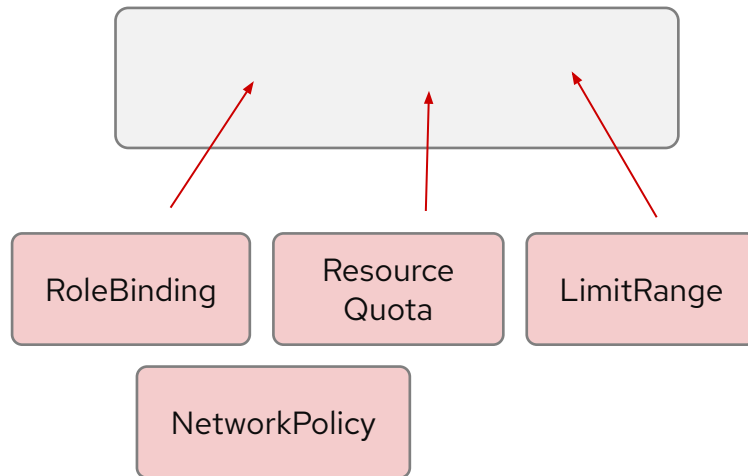
- ▶ Project (= Namespace の拡張) を実装。
- ▶ デフォルトで Project を作成する時に、リソースの制限などの設定をテンプレートのよう適用
 - **自動でNamespace 毎のリソースの使用制限の適用** ができる
 - マルチテナント管理として namespaceが管理しやすくなる
- ▶ この機能により、Project 作成時の管理者による Limit Range 等の個別制限適用の手間を省く事が可能
 - LimitRange: Pod や Container に割り当てるリソースの最大や最小値を設定する
 - ResourceQuota: Namespace に対するシステムリソースのクォータを設定する
 - NetworkPolicy: Namespace 内や Namespace をまたぐ Pod 間の通信を L4 で制御する
 - RBAC: ロールベース (RBAC) のアクセス管理を提供する

プロジェクト・リクエスト・テンプレートとクォータ / 制限

namespace の作成

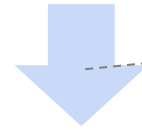


制限の無い namespace



個別に必要な設定を手
動で適用

project の作成



template の
自動適用

基本設定のされた project

- project の 作成ユーザー名を annotation に銘記
- project の 作成ユーザーに、デフォルトで用意してあるadmin ClusterRole に紐付け (RoleBinding)
- Resource Quota の適用
- LimitRange の適用
- NetworkPolicy の適用
- etc...

制限が設定された
project/namespaceを用
意できる

```
....
metadata:
  creationTimestamp: null
  name: admin
  namespace: ${PROJECT_NAME}
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: admin
  subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: User
    name: ${PROJECT_ADMIN_USER}
- apiVersion: v1
  kind: ResourceQuota
  metadata:
    name: project-quota
    namespace: ${PROJECT_NAME}
  spec:
    hard:
      count/pods: 10
      limits.cpu: 6
      limits.memory: 16Gi
      requests.cpu: 4
      requests.memory: 8Gi
      requests.storage: 20G
- apiVersion: v1
  kind: LimitRange
  metadata:
    name: project-limits
    namespace: ${PROJECT_NAME}
  spec:
    limits:
      - default:
          cpu: 1
          memory: 1Gi
        defaultRequest:
          cpu: 500m
          memory: 500Mi
        type: Container
- apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: allow-from-openshift-ingress
    namespace: ${PROJECT_NAME}
  spec:
    ingress:
      - from:
```


プロジェクト・リクエスト・テンプレートとクォータ / 制限

プロジェクト内で使用するオブジェクト数やリソース(CPU,メモリ等)を制限するテンプレートを設定します

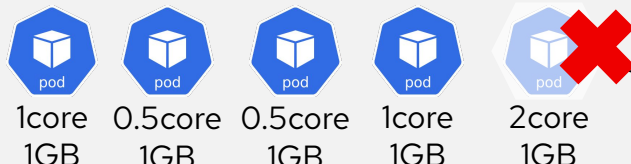
oc new-projectを利用するとデフォルトのTemplateを使用してプロジェクトを作成する(デフォルトではリソース制限なし)

プロジェクト作成

oc new-project

リソース制限を行うカスタム
テンプレートを使用

Project
template-test
(リソース制限のある project)



ResourceQuota

プロジェクトごとの総リソース消費量を制限

```
- apiVersion: v1
kind: ResourceQuota
metadata:
  name: ${PROJECT_NAME}-quota
spec:
  hard:
    pods: 10
    requests.cpu: 4000m
    requests.memory: 8Gi
    resourcequotas: 1
    requests.storage: 50Gi
    persistentvolumeclaims: 5
```

作成可能なPodの数
CPU総量
メモリ総量
ストレージ総量
PVC数

ResourceQuotaの制約で
CPUは合計4coreまでなので追加で
2coreのPodはデプロイ不可

LimitRange

プロジェクト内のリソース毎の消費量を指定

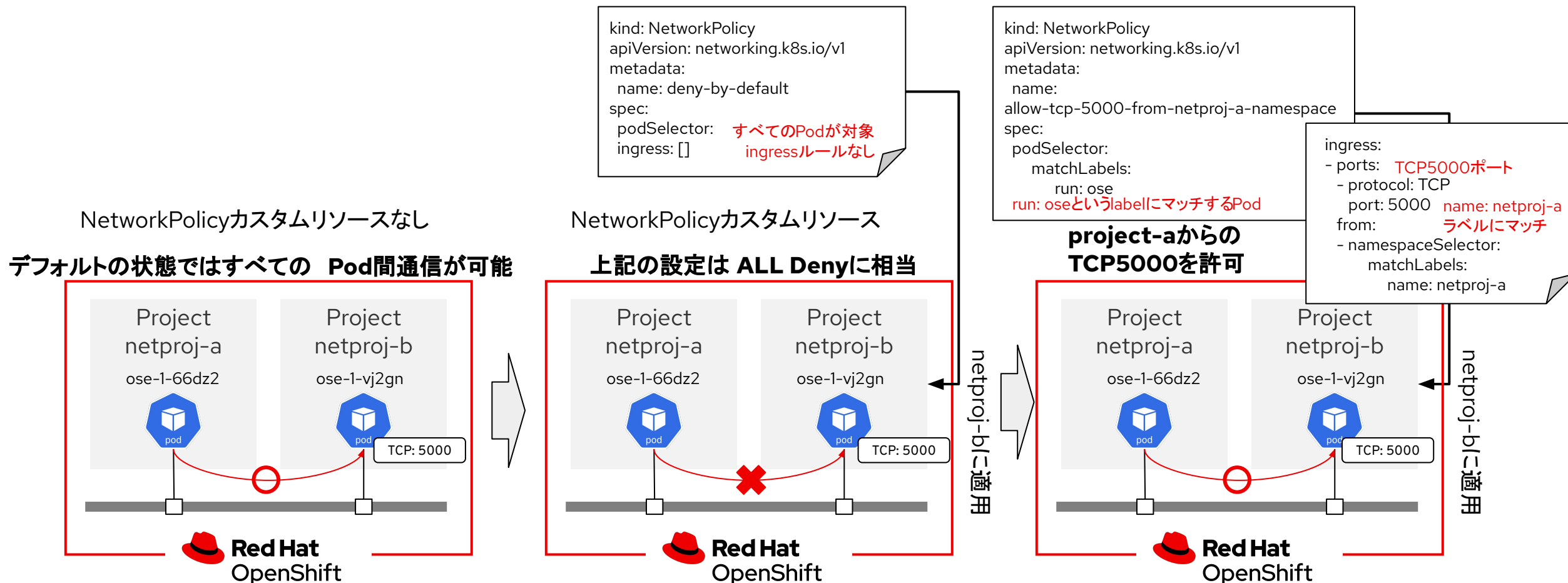
```
- apiVersion: v1
kind: LimitRange
metadata:
  name: ${PROJECT_NAME}-limits
  creationTimestamp: null
spec:
  limits:
    -
      type: Container
      max:
        cpu: 4000m
        memory: 1024Mi
      min:
        cpu: 10m
        memory: 5Mi
      default:
        cpu: 4000m
        memory: 1024Mi
      defaultRequest:
        cpu: 100m
        memory: 512Mi
```

limitやrequestに指定できる最大値
limitやrequestに指定できる最小値
指定がない場合の最大値
指定がない場合の最小値



OpenShift の Network Policy ベースの SDN

NetworkPolicyカスタムリソースを利用することでプロジェクト間のネットワークを分離することができます



Projectのセルフプロビジョニングの無効化

- ・デフォルトでは認証済みのユーザはプロジェクトを作成することが可能です
- ・プロジェクトのセルフプロビジョニングを無効化することができます

ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  creationTimestamp: "2022-06-07T02:52:33Z"
  name: self-provisioners
  resourceVersion: "9988"
  uid: 24f23fdd-4a10-40f2-8ca2-0068d4b51a8a
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: self-provisioner
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: system:authenticated:oauth
```

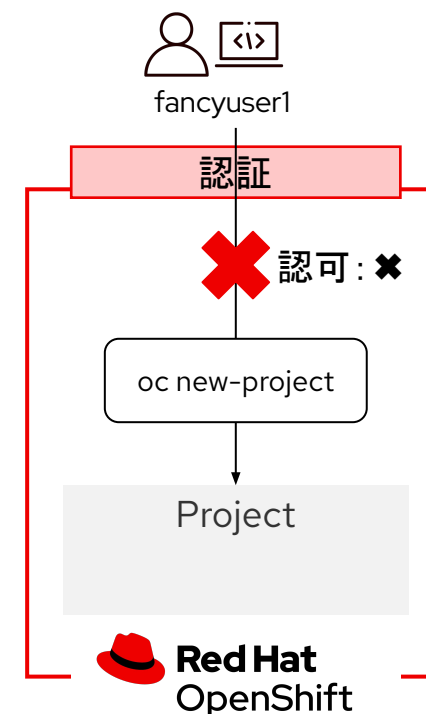
認証済みのユーザが属するグループは、self-provisioners roleにバインドされているためセルフプロビジョニングが可能

ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "false"
  creationTimestamp: "2022-06-07T02:52:33Z"
  name: self-provisioners
  resourceVersion: "66495"
  uid: 24f23fdd-4a10-40f2-8ca2-0068d4b51a8a
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: self-provisioner
```

oc patchコマンドで設定変更

```
oc patch clusterrolebinding.rbac self-provisioners -p '{"subjects": null}'
oc patch clusterrolebinding.rbac self-provisioners -p '{"metadata": {"annotations": {"rbac.authorization.kubernetes.io/autoupdate": "false"} } }'
```

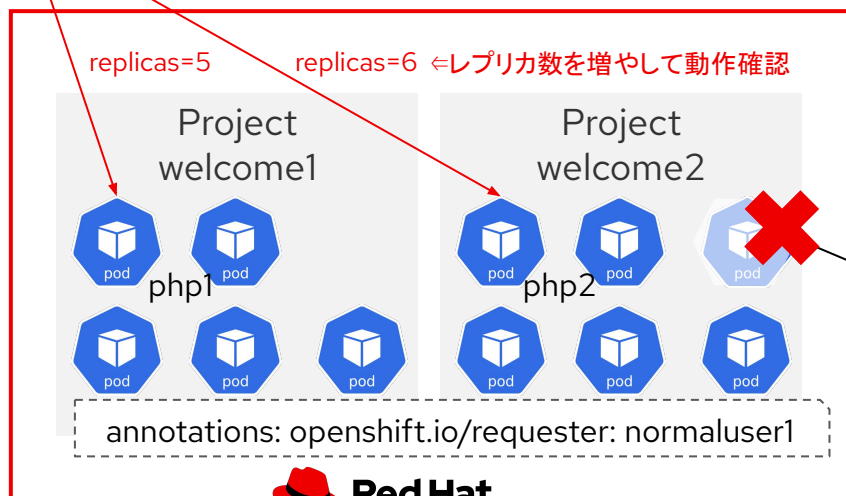


クラスタリソースのクォータ

・特定のユーザにクォータを設定します

例) 特定のユーザに対して Pod 数を10以下に制限する
clusterresourcequotaを作成します
openshift.io/requester=のannotationキーを利用します

※管理者権限で
normaluser1 `oc create clusterquota for-user-normaluser1 \`
`--project-annotation-selector openshift.io/requester=normaluser1 \`
`--hard pods=10`



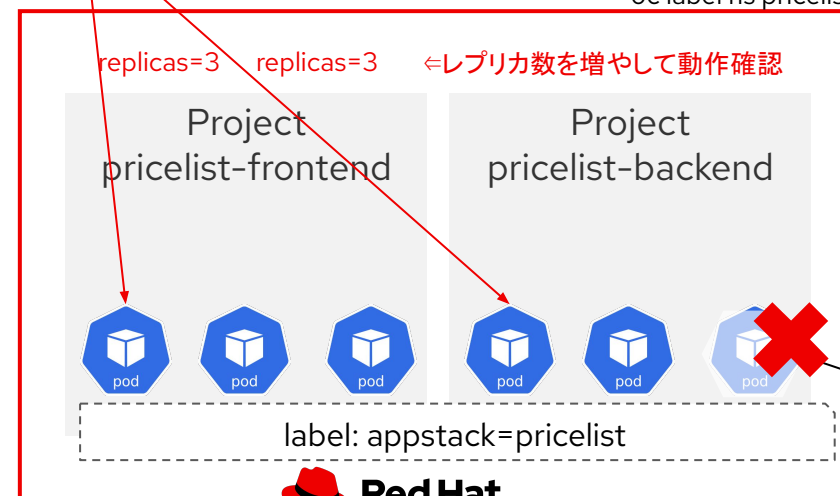
pod数が10を超えるため1個作成されない



・labelを利用してアプリケーションにクォータを設定します

例) 特定のアプリに対して Pod 数を5以下に制限する
clusterresourcequotaを作成します
labelを利用します

※管理者権限で
normaluser1 `oc create clusterresourcequota for-pricelist \`
`--project-label-selector=appstack=pricelist \`
`--hard=pods=5`
ネームスペースにlabelを付けておく
`oc label ns pricelist-frontend appstack=pricelist`
`oc label ns pricelist-backend appstack=pricelist`



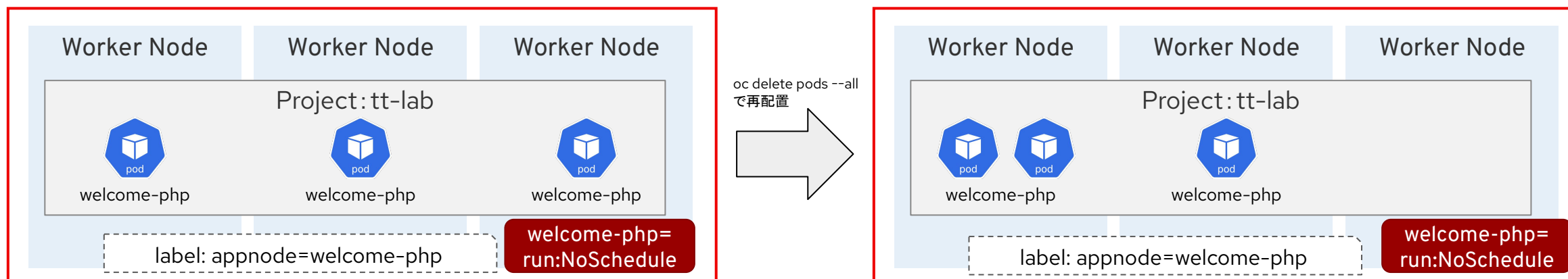
pod数が5を超えるため1個作成されない



Taint と Toleration

Taint と Toleration は、協調して動作し、ワークロードが他のノードにスケジュールされないようにします

1つのNodeにTaint設定を行い、Podを再配置すると、Tolerationの設定がないため、TaintされたNodeではPodが起動しません



welcome-phpのdeploymentにTolerationの設定して、Podを再配置します

welcome-php=run:NoSchedule

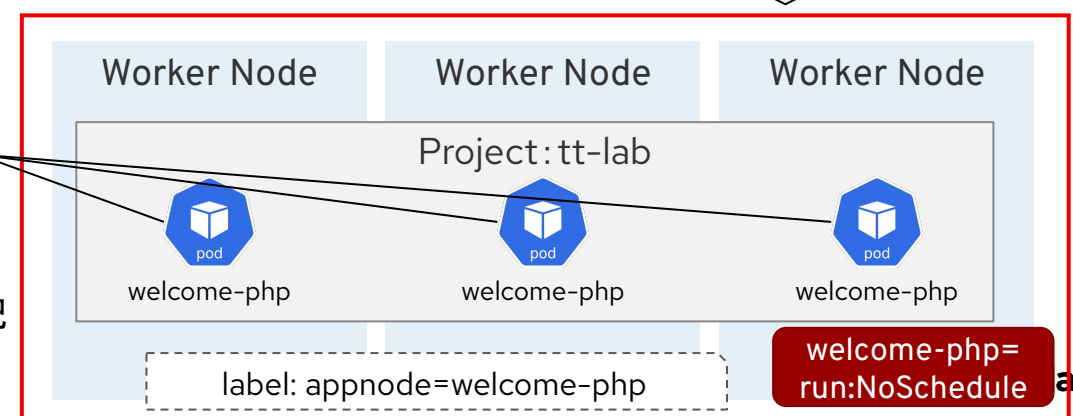
welcome-php:runにマッチしないPodは配置させないTaint設定

Taintの種類

key=value:NoSchedule
key=value:NoExecute
key=value:PreferNoSchedule

tolerations:
- key: "welcome-php"
 operator: "Equal"
 value: "run"
 effect: "NoSchedule"

Taint設定されたNodeにもPodが配置されます





クロージング

17:15 - 17:30

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.

